PROYECTO FINAL - SISPRO3D

Diplomado Desarrollo de Sistemas con Tecnologías Java

Documento de Especificación Técnica y Arquitectura

1. Presentación

- **Nombre del proyecto:** SISPRO3D
- **Alumno:** Axel Fernando Montiel Aviles
- **Fecha:** 15 de Octubre de 2026
- **Versión del documento:** 1.0

2. Resumen Ejecutivo

Aplicación web enfocada en fungir como intermediaria directa para conectar a usuarios que requieren servicios específicos de modelados 3D con expertos independientes, permitiendo la búsqueda, solicitud, cotización, seguimiento de órdenes de servicio y calificación de los trabajos prestados.

Alcance del proyecto

El sistema se limitará estrictamente a la intermediación y gestión del catálogo de servicios de modelado 3D, la gestión de cotizaciones, el flujo de solicitudes de trabajo y la evaluación de estas. Solo permitirá la intermediación directa sin gestionar nóminas internas ni retención de impuestos complejos.

3. Requerimientos Funcionales

Aquí se describirán los requerimientos funcionales esenciales del proyecto, como pueden ser los casos de uso, CRUDs, funcionalidades de negocio, etc. Así como una primera sección para definir el alcance de este proyecto.



Tabla de requerimientos funcionales mínimos

Requerimiento	Descripción
Gestión del Catálogo de Servicios	Permitir la administración de la información de los catálogos de servicios por medio de funcionalidades de Alta, Baja, Cambio y Consulta (CRUD).
Gestión de Usuarios	Permitir la administración de la información de los usuarios (clientes y expertos) por medio de funcionalidades de Alta, Baja, Cambio y Consulta (CRUD).
Gestión de Cotizaciones	Permitir a los clientes solicitar, recibir y gestionar cotizaciones personalizadas de los expertos para sus proyectos de modelado 3D.
Órdenes de Servicio	Permitir registrar y gestionar el flujo de las órdenes de servicio (solicitud, en proceso, completada, cancelada).
Sistema de Calificaciones y Reseñas	Permitir a los clientes calificar y reseñar el servicio una vez que la orden esté completada para garantizar control y transparencia.
Reportes y Estadísticas	Generar reportes sobre los servicios más solicitados y el desempeño de los expertos para apoyar la toma de decisiones.
Gestión de Archivos 3D	Permitir la carga y descarga segura de archivos de modelado 3D (.OBJ, .FBX, .STL, .BLEND) y sus recursos asociados (texturas, materiales) vinculados a una orden activa.
Previsualización de Trabajos	Permitir a los expertos subir imágenes de previsualización (renders) para que el cliente evalúe el trabajo antes de descargar los archivos fuente.
Control de Acceso a Entregables	Restringir el acceso a los archivos finales para garantizar que solo el cliente y el experto vinculados a la orden puedan visualizarlos y descargarlos.
Control de Perfiles y Autenticación	Delimitar las funciones de las personas usuarias mediante roles (Administrador, Cliente, Experto). El registro requerirá nombre, correo electrónico, perfil y contraseña.

Axel Fernando Montiel Aviles
Universidad Nacional Autónoma de México

Casos de uso típicos de el sistema

Actores del Sistema

- **Administrador:** Usuario encargado de la administración general de la plataforma.
- **Cliente:** Usuario solicitante del servicio de modelado 3D.
- **Experto:** Usuario proveedor del servicio de modelado 3D.

Casos de Uso por Módulo

1. Módulo de Usuarios y Seguridad

- Registrar un nuevo usuario en el sistema proporcionando su nombre, correo electrónico, perfil y contraseña.
- Iniciar sesión mediante un control de acceso con autenticación y autorización según el perfil del usuario.
- Administrar la información de los usuarios mediante funcionalidades de alta, baja, cambio y consulta.

2. Módulo de Catálogo de Servicios

- Administrar la información de los catálogos de servicios mediante funcionalidades de alta, baja, cambio y consulta.
- Buscar servicios específicos en la plataforma utilizando un motor de búsqueda con filtros.

3. Módulo de Cotizaciones

- Solicitar cotizaciones personalizadas detallando los requerimientos del proyecto de modelado 3D.
- Responder a las solicitudes de los clientes enviando propuestas de costos y tiempos de entrega.
- Recibir y gestionar las cotizaciones personalizadas enviadas por los expertos.

4. Módulo de Gestión de Órdenes

- Registrar una nueva solicitud de servicio.
- Gestionar y dar seguimiento al flujo de las órdenes de servicio desde su creación hasta su conclusión.
- Actualizar el estado de la orden de servicio.
- Generar reportes de actividad.

5. Módulo de Entregas y Revisión

- Subir imágenes de previsualización (renders).
- Evaluar el trabajo a través de los renders.
- Aprobar el modelo 3D o solicitar modificaciones.
- Cargar y descargar de forma segura archivos de modelado 3D finales.

Axel Fernando Montiel Aviles
Universidad Nacional Autónoma de México

6. Módulo de Evaluaciones

- Calificar y registrar una reseña del servicio.
- Calcular de forma automatizada la calificación promedio.

4. Requerimientos No Funcionales

En esta sección explicaremos algunos de los requerimientos no funcionales que implica el desarrollo del proyecto.

Tabla de requerimientos no funcionales

Requerimiento	Descripción
Tecnologías del Sistema	El sistema será una aplicación web utilizando Java y el ecosistema de Spring (Spring Boot, Spring Data JPA, Spring Security, etc.).
Base de Datos	Se utilizará una base de datos relacional para almacenar los datos de la aplicación web.
Tiempo de Respuesta	Deberá contar con un tiempo de respuesta menor a 5 segundos para cada transacción.
Interfaz e Idioma	La interfaz web deberá ser "Responsive" para poder ejecutarse correctamente en todo tipo de pantallas utilizando solo el idioma español.
Seguridad de Datos	Encriptación de contraseñas.
Disponibilidad	Disponibilidad 24/7.

Seguridad, Validaciones y Buenas Prácticas

Categoría	Descripción
Seguridad (Autenticación)	Se implementará seguridad basada en tokens (JWT y Refresh Token) para gestionar las sesiones de los usuarios, protegiendo las rutas y vistas (Thymeleaf) de la aplicación web según el rol (Administrador, Cliente, Experto).
Seguridad (Contraseñas)	Uso de algoritmos de hash seguros al implementar la encriptación de contraseñas en la base de datos.

Categoría	Descripción
Validaciones	El backend implementará validaciones a nivel de controlador para asegurar que los datos ingresados desde los formularios cumplen con los formatos, longitudes y tipos requeridos.
Buenas Prácticas (MVC y Vistas)	La arquitectura se basará en el patrón Modelo-Vista-Controlador (MVC) utilizando **Thymeleaf** para el renderizado de vistas dinámicas en el servidor, integrando el paso seguro del JWT en las peticiones e implementando el patrón DTO para la transferencia de datos hacia las vistas.
Mantenibilidad	Se implementará un manejador global de excepciones para interceptar errores de forma centralizada y redirigir al usuario a páginas de error personalizadas y estandarizadas en lugar de mostrar errores genéricos del servidor.

5. Arquitectura del Sistema

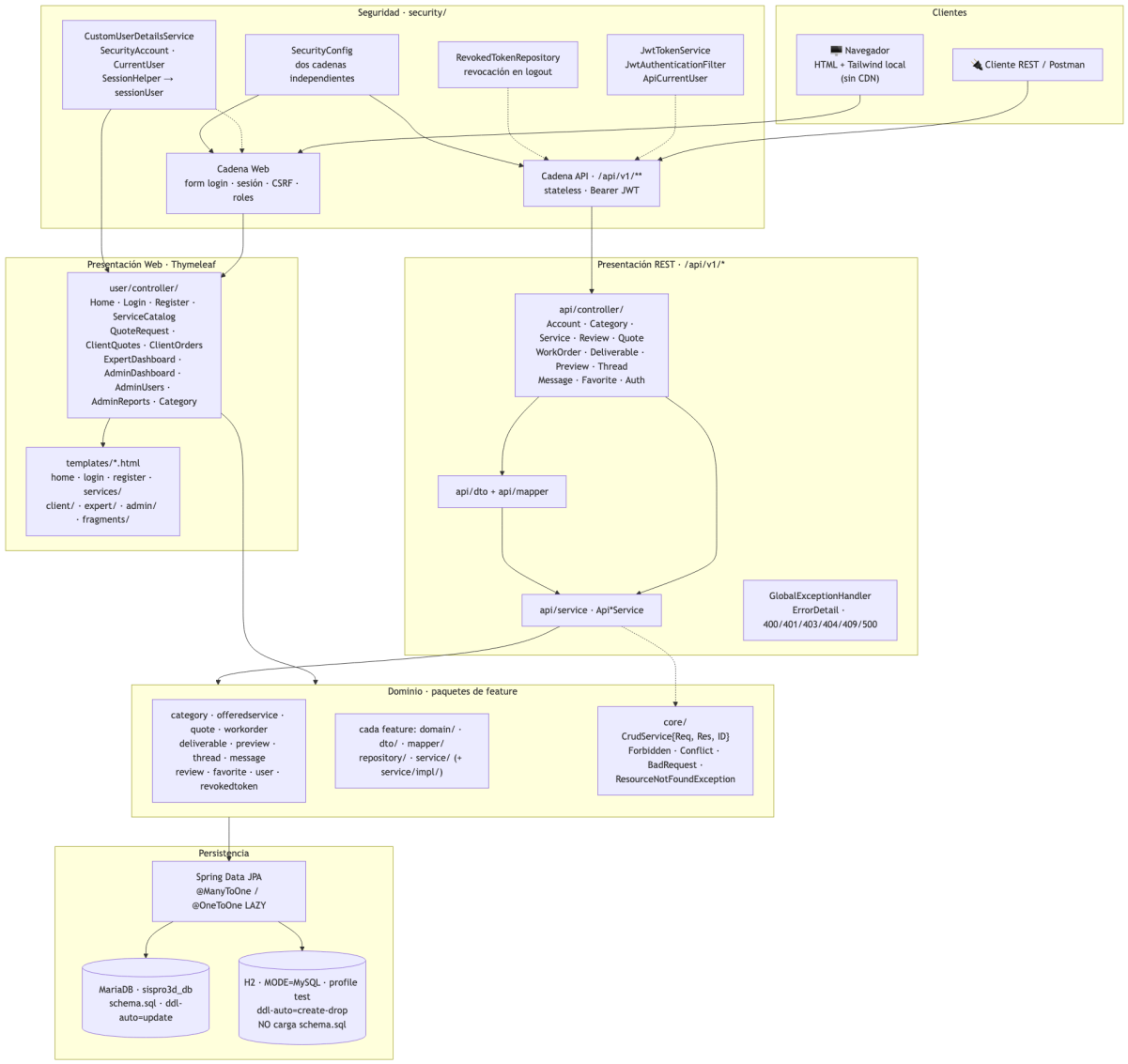


Figura: s05-arquitectura-capas

El diagrama de arquitectura refleja una estructura por capas alineada con las buenas prácticas de Spring Framework, diseñada para soportar una doble modalidad de interacción (interfaz web tradicional renderizada en servidor y API REST sin estado):

- **Capa de Seguridad (SecurityConfig):** Implementa dos cadenas de seguridad independientes (SecurityFilterChain):
    - **Cadena Web:** Orientada a la navegación por navegador, utiliza autenticación basada en formularios (`formLogin`), manejo de sesiones HTTP, protección contra ataques CSRF y control de acceso por roles mediante `CustomUserDetailsService`.
    - **Cadena API (`/api/v1/**`):** Configurada de forma sin estado (`STATELESS`), prescinde de sesiones HTTP y valida cada petición de manera independiente mediante tokens de acceso `Bearer JWT` procesados por el filtro `JwtAuthenticationFilter` y apoyados en

`JwtTokenService` y `RevokedTokenRepository` para el control de invalidación en cierres de sesión.

- **Capa de Presentación:** Dividida en dos frentes paralelos:
  - **Presentación Web (Thymeleaf):** Controladores MVC (`user/controller/*`) que gestionan la interacción directa con las vistas HTML dinámicas y plantillas de fragmentos.
  - **Presentación REST (`/api/v1/*`):** Controladores de API (`api/controller/*`) que exponen recursos en formato JSON, apoyándose en la capa de transformación `DTO + Mapper` y en servicios especializados (`ApiService`) para delegar el procesamiento, además de un gestor global de excepciones (`GlobalExceptionHandler`) para estandarizar las respuestas de error (400, 401, 403, 404, 409, 500).
- **Capa de Dominio:** Organizada modularmente por funcionalidades o *features* (como `category`, `offeredservice`, `quote`, `workorder`, `deliverable`, `review`, entre otros). Cada módulo encapsula sus entidades de dominio, DTOs, mappers, repositorios e implementaciones de servicio (`service/impl`), promoviendo el acoplamiento débil y la alta cohesión.
- **Capa de Persistencia:** Utiliza Spring Data JPA con mapeos relacionales optimizados (relaciones `@ManyToOne` y `@OneToOne` configuradas con carga perezosa `LAZY`). La arquitectura permite alternar de forma transparente entre un motor de base de datos relacional **MariaDB** para entornos de producción/desarrollo y una base de datos en memoria **H2** con modo de compatibilidad MySQL para la ejecución de pruebas unitarias e integradas sin persistencia permanente.

6. Modelo de Datos

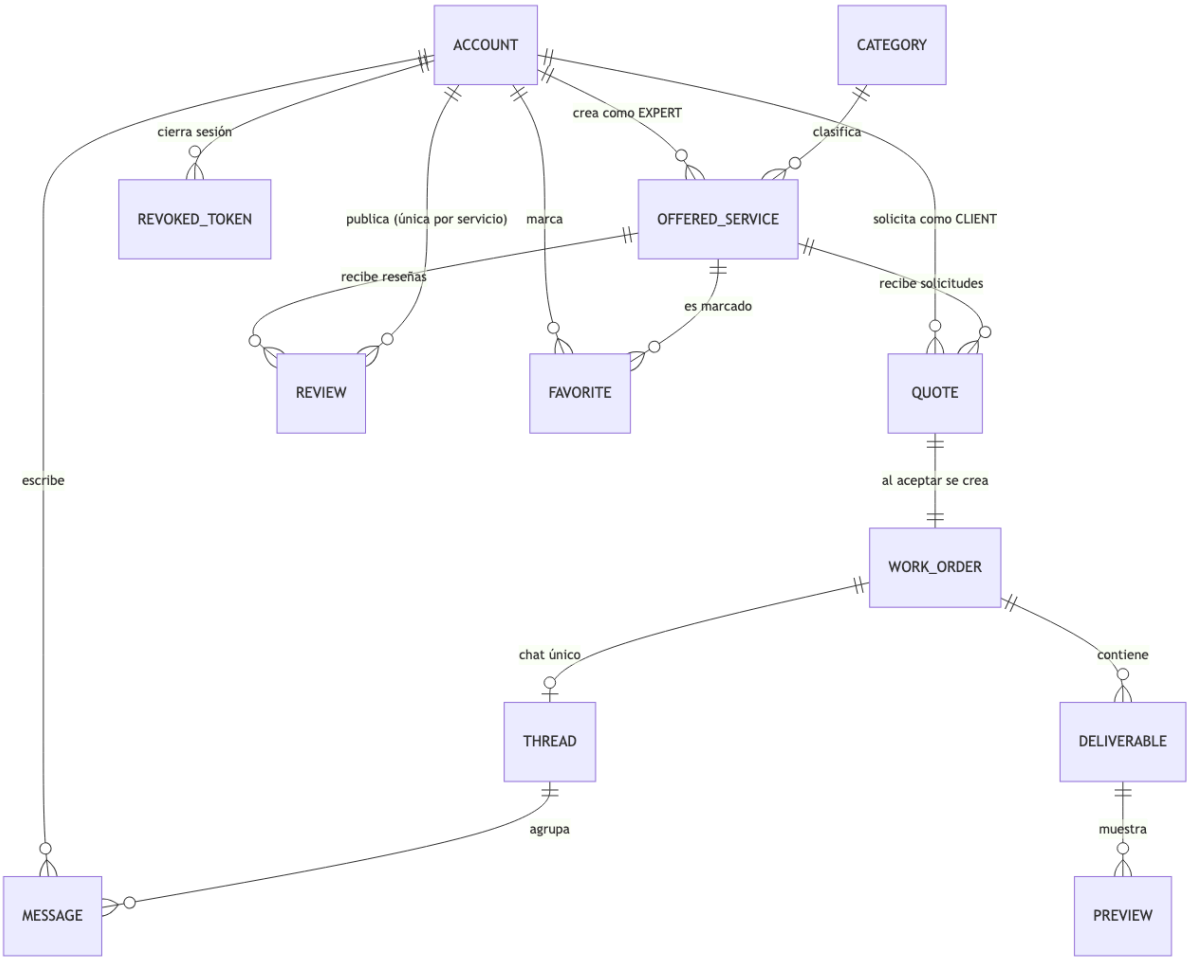


Figura: s06-modelo-datos

El diagrama entidad-relación define el modelo conceptual y relacional de la base de datos, estructurando las entidades principales y regulando sus interacciones mediante restricciones y reglas de negocio específicas:

- **Gestión de Usuarios y Catálogo (`ACCOUNT`, `CATEGORY`, `OFFERED_SERVICE`):** La entidad **`ACCOUNT`** representa a todos los usuarios del sistema. Un usuario con rol `EXPERT` puede crear múltiples servicios en **`OFFERED_SERVICE`** (relación 1:N), mientras que cada servicio está clasificado bajo una única **`CATEGORY`** (1:N). Un usuario con rol `CLIENT` puede marcar varios servicios como preferidos a través de la entidad intermedia **`FAVORITE`** (relación N:M).
- **Flujo de Cotización y Trabajo (`QUOTE`, `WORK_ORDER`):** Un cliente (`CLIENT`) puede solicitar cotizaciones (`QUOTE`) sobre un servicio expuesto. Cada **`QUOTE`** pertenece a una cuenta y a un servicio ofertado (1:N). Cuando una cotización es aceptada por el cliente, se genera automáticamente una orden de trabajo (**`WORK_ORDER`**) vinculada de manera directa (relación 1:1).
- **Entregables, Previsualizaciones y Comunicación (`DELIVERABLE`, `PREVIEW`, `THREAD`, `MESSAGE`):** Una orden de trabajo (`WORK_ORDER`) puede contener múltiples

archivos entregables finales en **DELIVERABLE** (1:N), y cada entregables puede incluir previsualizaciones o renders asociados en **PREVIEW** (1:N). Adicionalmente, cada WORK_ORDER posee un único hilo de conversación o chat especializado en **THREAD** (1:1), el cual agrupa múltiples mensajes (**MESSAGE**, 1:N) redactados por el cliente o el experto involucrados.

- **Reseñas y Seguridad (REVIEW, REVOKED_TOKEN):** Una vez completada la orden de trabajo (WORK_ORDER), el cliente puede publicar una evaluación en **REVIEW** asociada al servicio (**OFFERED_SERVICE**) y a la cuenta del experto. Por otro lado, la entidad **REVOKED_TOKEN** almacena los identificadores de tokens JWT invalidados cuando una cuenta (**ACCOUNT**) cierra sesión explícitamente.
- **Reglas de Negocio y Restricciones de Estado:**
    - **Estados de Servicio (ServiceStatus):** Un servicio puede encontrarse en estados PENDING (esperando aprobación del administrador), APPROVED (visible en el catálogo público) o REJECTED.
    - **Estados de Cotización (QuoteStatus):** Inicia en PENDING al ser solicitada, transitando a ACCEPTED (creando la orden de trabajo), REJECTED o EXPIRED (al superar la fecha límite fijada en validUntil).
    - **Estados de Orden de Trabajo (WorkOrderStatus):** Controla el flujo del proyecto transitando secuencialmente por PENDING > IN_PROGRESS > IN_REVIEW > COMPLETED (o CANCELED si es rescindida por el cliente), garantizando que solo las órdenes en estado COMPLETED habiliten la creación de una reseña.

7. Diseño del Sistema

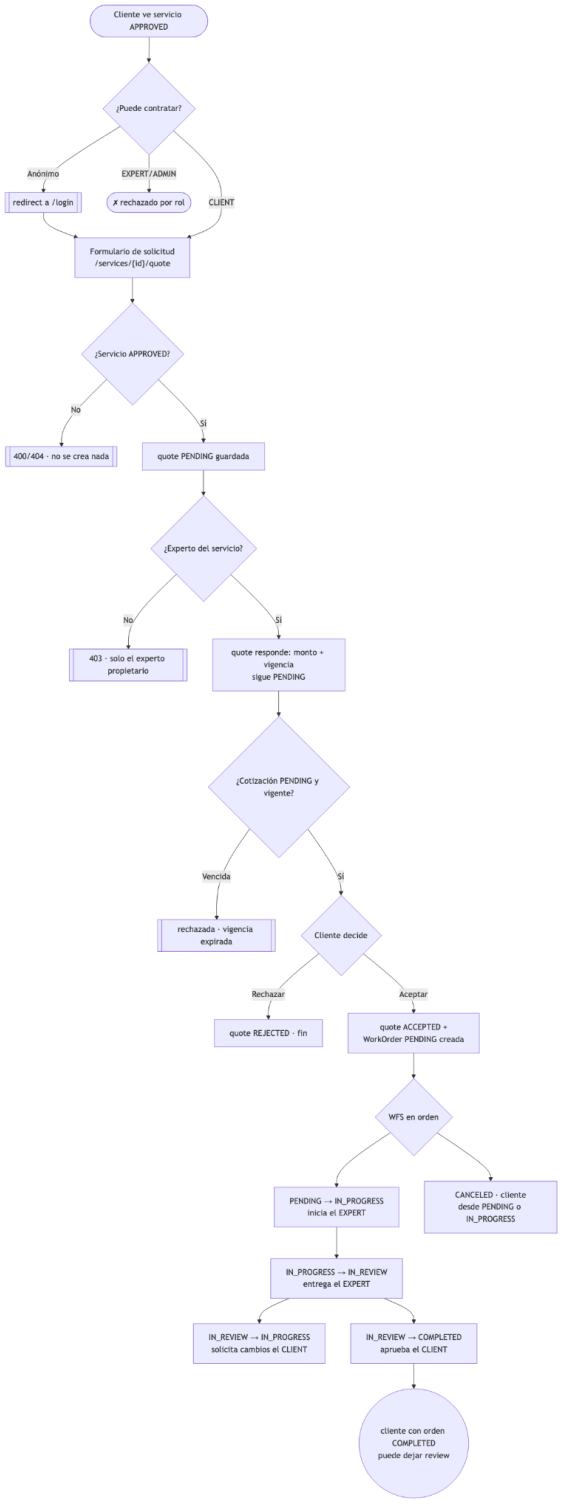


Figura: s07-a-actividad-contratacion

- **Diagrama de Actividad - Ciclo de Contratación (s07-actividad-contratacion):** Modela el flujo de trabajo completo de punta a punta desde la perspectiva de la lógica de negocio. Muestra la interacción entre los distintos roles (Cliente, Experto y Administrador) y las validaciones del sistema. Describe las

restricciones de acceso por rol, la verificación de disponibilidad e idoneidad del servicio (estado APPROVED), la generación y respuesta de la cotización, la transición condicional tras la aceptación/rechazo, el ciclo de ejecución de la orden con sus iteraciones de entrega y revisión, hasta la finalización que habilita la creación de una reseña.

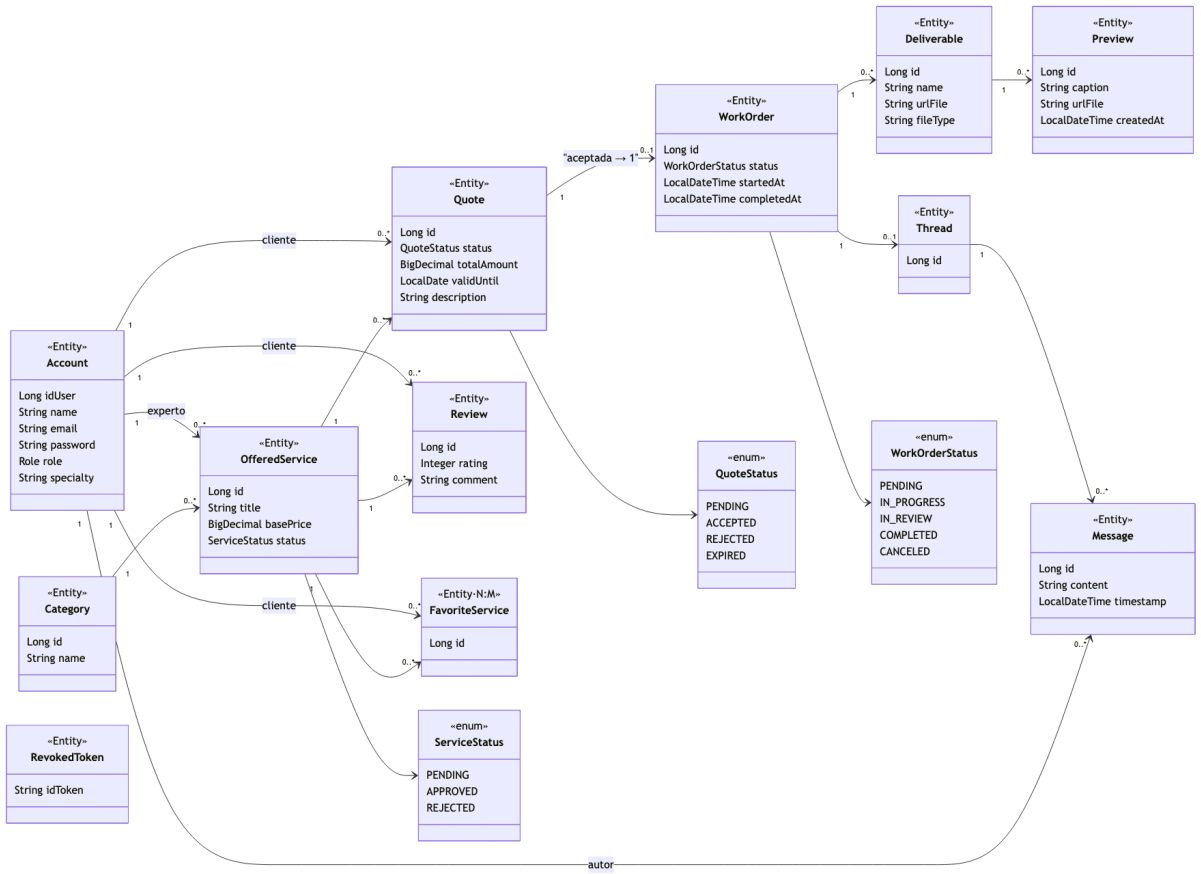


Figura: s07-b-clases-entidades-dominio

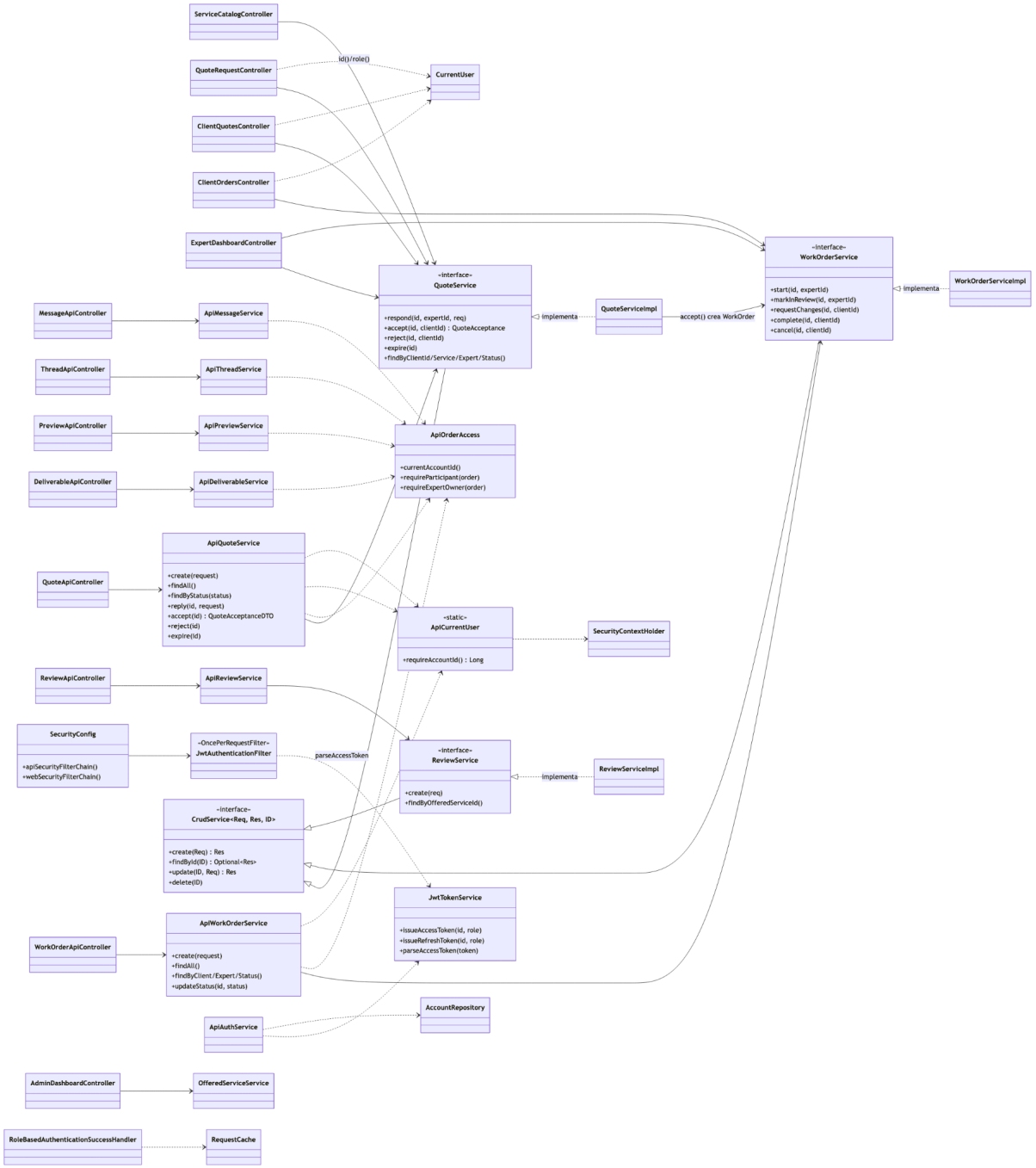


Figura: s07-c-clases-servicios-controladores

- **Diagrama de Clases - Entidades de Dominio (s07-b-clases-entidades-dominio):** Representa la estructura de objetos de dominio persistence-mapped mediante anotaciones JPA. Muestra las clases de entidad clave (`Account`, `OfferedService`, `Quote`, `WorkOrder`, `Deliverable`, `Preview`, `Thread`, `Message`, `Review`, `Category`, `RevokedToken`), sus atributos principales, así como las enumeraciones de estado (`ServiceStatus`, `QuoteStatus`, `WorkOrderStatus`) y sus relaciones de asociación y multiplicidad.
- **Diagrama de Clases - Servicios y Controladores (s07-c-clases-servicios-controladores):** Detalla la arquitectura de componentes lógicos de la aplicación. Ilustra la interacción entre la capa web

MVC y REST Controllers (`ServiceCatalogController`, `QuoteApiController`, `WorkOrderApiController`, etc.), los servicios de la capa de aplicación e interfaces de negocio (`QuoteService`, `WorkOrderService`, `ReviewService`, `JwtTokenService`), y su conexión con los componentes de seguridad (`SecurityConfig`, `JwtAuthenticationFilter`, `ApiCurrentUser`) y repositorios de persistencia.

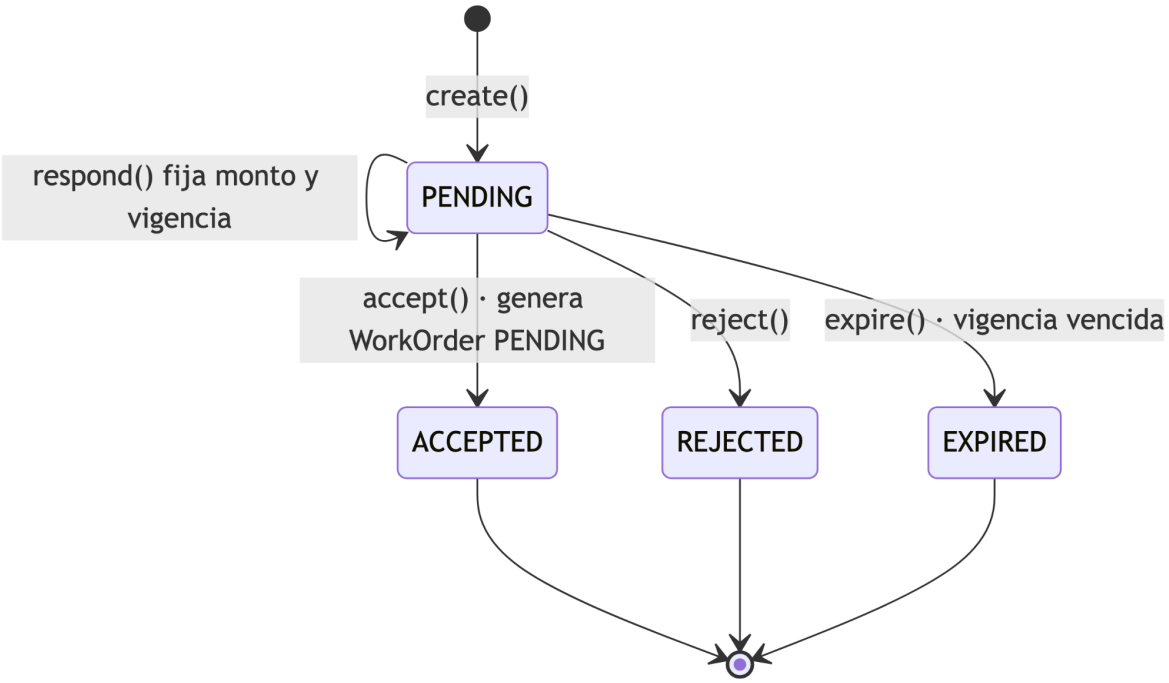


Figura: s07-d-estado-cotizacion

- **Máquina de Estados de Cotización (s07-estado-cotizacion):** Describe el ciclo de vida de una solicitud de cotización (`Quote`). Inicia en estado `PENDING` cuando el cliente la registra o el experto actualiza su propuesta (monto y vigencia). Transiciona a `ACCEPTED` cuando el cliente aprueba la propuesta (desencadenando la creación de la orden de trabajo), a `REJECTED` si el cliente decide declinarla, o a `EXPIRED` de forma automatizada cuando la fecha límite fijada en `validUntil` es superada sin confirmación.

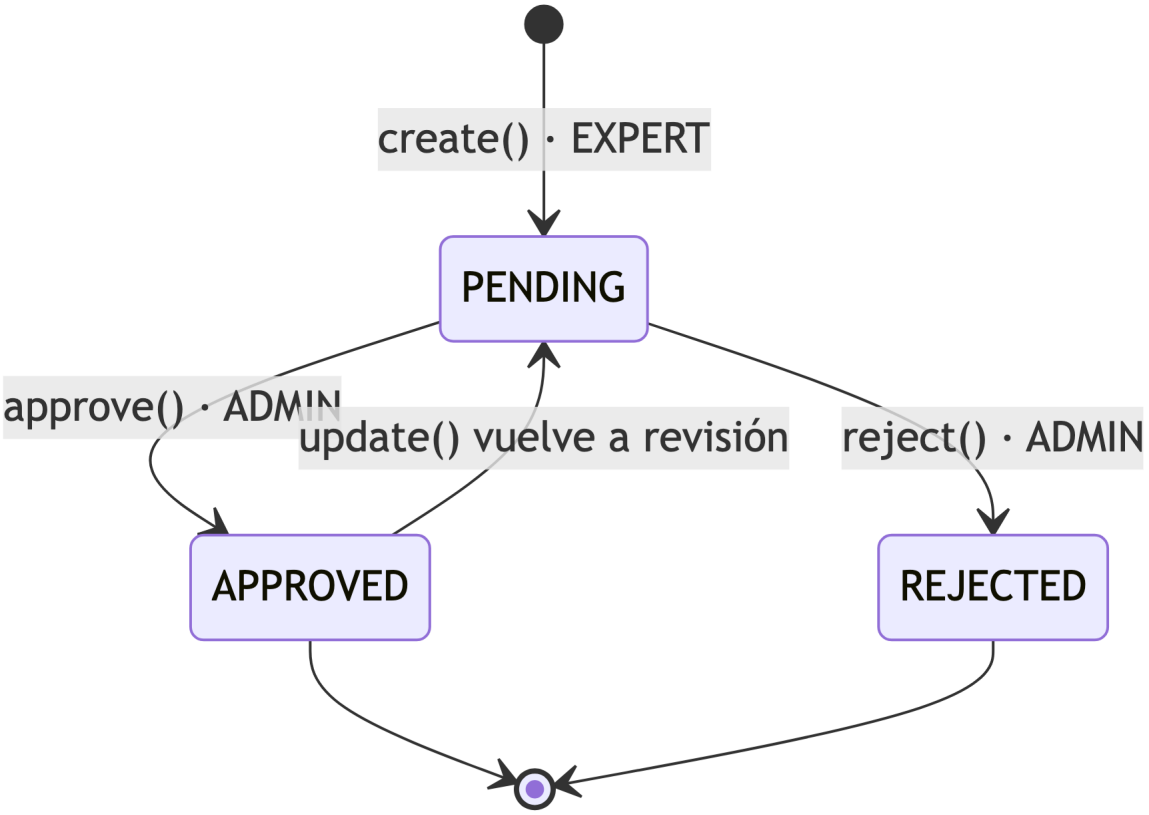


Figura: s07-e-estado-servicio

- **Máquina de Estados de Servicio Ofrecido (s07-estado-servicio):** Modela el flujo de aprobación administrativa para los servicios creados por los expertos (`OfferedService`). Al crearse, un servicio ingresa en estado `PENDING` hasta ser evaluado por un Administrador, quien puede transicionarlo a `APPROVED` (haciéndolo visible en el catálogo público para contratación) o a `REJECTED`. Si un experto edita un servicio aprobado, este vuelve a estado `PENDING` para requerir una nueva revisión de administración.

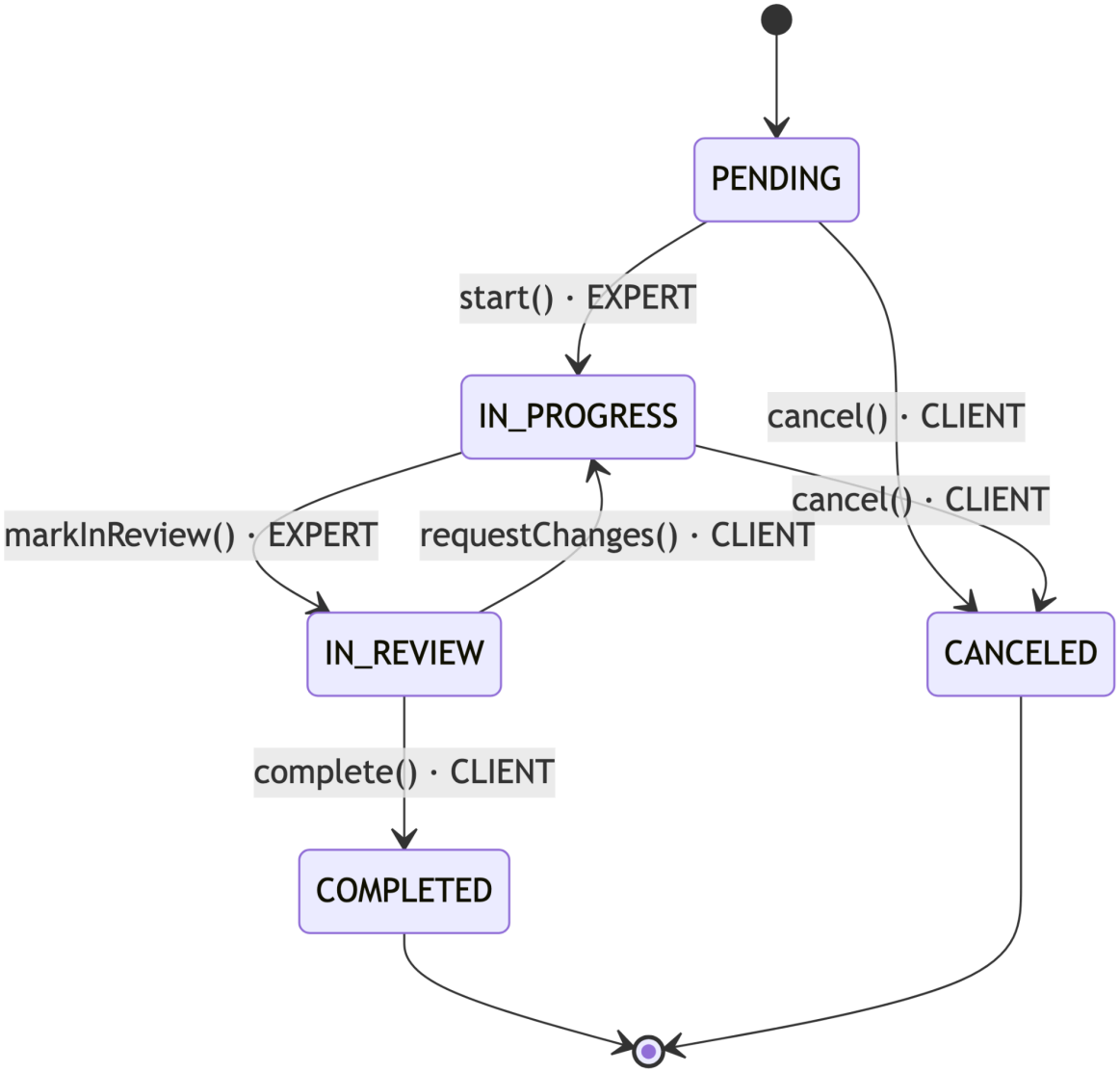


Figura: s07-f-estado-workorder

- **Máquina de Estados de Orden de Trabajo (s07-estado-workorder):** Regula la ejecución del proyecto (`WorkOrder`). Inicia en `PENDING` tras aceptarse la cotización; pasa a `IN_PROGRESS` cuando el experto inicia la labor; transiciona a `IN_REVIEW` cuando el experto entrega avances/renders; puede retornar a `IN_PROGRESS` si el cliente solicita modificaciones, o avanzar a `COMPLETED` si el cliente aprueba el entregable. Asimismo, permite la cancelación a estado `CANCELED` por parte del cliente únicamente desde los estados `PENDING` o `IN_PROGRESS`.

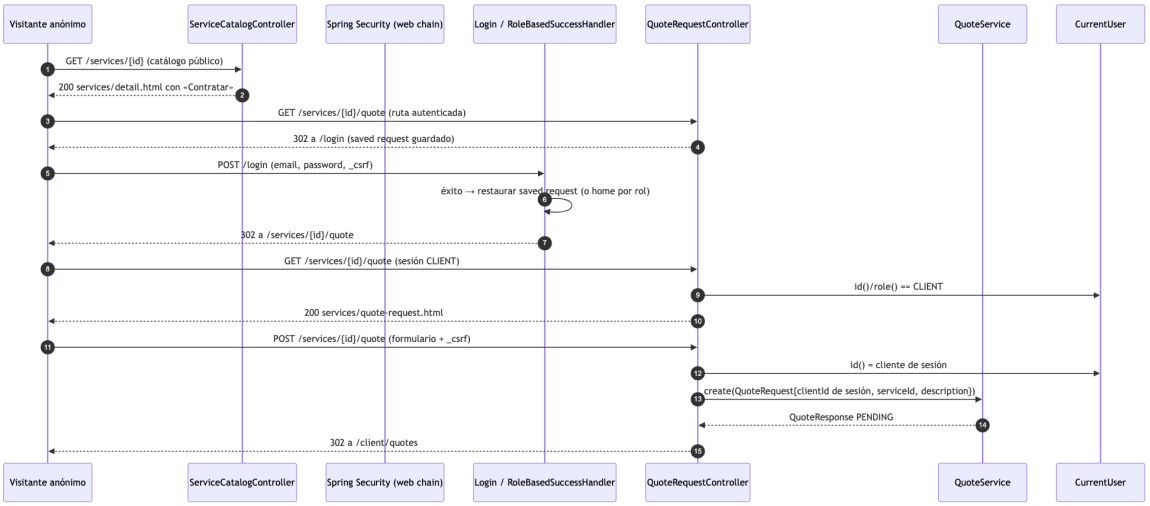


Figura: s07-g-secuencia-catalogo-web

- **Flujo Web de Solicitud de Cotización (s07-secuencia-catalogo-web):** Ilustra la navegación e interacción desde la perspectiva de un visitante anónimo que consulta el catálogo público (`GET /services/{id}`). Al intentar solicitar una cotización (`GET /services/{id}/quote`), Spring Security intercepta la petición no autenticada, guarda la solicitud en sesión y redirige al usuario hacia `/login`. Una vez que el usuario ingresa sus credenciales (`POST /login`) y `RoleBasedSuccessHandler` valida el inicio de sesión, se restaura la petición guardada redirigiendo automáticamente de vuelta a `/services/{id}/quote`. Con la sesión activa del rol `CLIENT`, el controlador renderiza el formulario (`quote-request.html`), procesa el envío (`POST /services/{id}/quote`) y delega a `QuoteService` la creación de la cotización en estado `PENDING`, redirigiendo finalmente al cliente a su panel de cotizaciones (`/client/quotes`).
- **Flujo Web de Solicitud de Cotización (s07-secuencia-catalogo-web):** Ilustra la navegación e interacción desde la perspectiva de un visitante anónimo que consulta el catálogo público (`GET /services/{id}`). Al intentar solicitar una cotización (`GET /services/{id}/quote`), Spring Security intercepta la petición no autenticada, guarda la solicitud en sesión y redirige al usuario hacia `/login`. Una vez que el usuario ingresa sus credenciales (`POST /login`) y `RoleBasedSuccessHandler` valida el inicio de sesión, se restaura la petición guardada redirigiendo automáticamente de vuelta a `/services/{id}/quote`. Con la sesión activa del rol `CLIENT`, el controlador renderiza el formulario (`quote-request.html`), procesa el envío (`POST /services/{id}/quote`) y delega a `QuoteService` la creación de la cotización en estado `PENDING`, redirigiendo finalmente al cliente a su panel de cotizaciones (`/client/quotes`).
- **Diagrama de Actividad - Ciclo de Contratación (s07-actividad-contratacion):** Modela el flujo de trabajo completo de punta a punta desde la perspectiva de la lógica de negocio. Muestra la interacción entre los distintos roles (Cliente, Experto y Administrador) y las validaciones del sistema. Describe las restricciones de acceso por rol, la verificación de disponibilidad e idoneidad del

servicio (estado `APPROVED`), la generación y respuesta de la cotización, la transición condicional tras la aceptación/rechazo, el ciclo de ejecución de la orden con sus iteraciones de entrega y revisión, hasta la finalización que habilita la creación de una reseña.

- **Máquina de Estados de Cotización (s07-estado-cotizacion):** Describe el ciclo de vida de una solicitud de cotización (`Quote`). Inicia en estado `PENDING` cuando el cliente la registra o el experto actualiza su propuesta (monto y vigencia). Transiciona a `ACCEPTED` cuando el cliente aprueba la propuesta (desencadenando la creación de la orden de trabajo), a `REJECTED` si el cliente decide declinarla, o a `EXPIRED` de forma automatizada cuando la fecha límite fijada en `validUntil` es superada sin confirmación.
- **Máquina de Estados de Servicio Ofrecido (s07-estado-servicio):** Modela el flujo de aprobación administrativa para los servicios creados por los expertos (`OfferedService`). Al crearse, un servicio ingresa en estado `PENDING` hasta ser evaluado por un Administrador, quien puede transicionarlo a `APPROVED` (haciéndolo visible en el catálogo público para contratación) o a `REJECTED`. Si un experto edita un servicio aprobado, este vuelve a estado `PENDING` para requerir una nueva revisión de administración.
- **Máquina de Estados de Orden de Trabajo (s07-estado-workorder):** Regula la ejecución del proyecto (`WorkOrder`). Inicia en `PENDING` tras aceptarse la cotización; pasa a `IN_PROGRESS` cuando el experto inicia la labor; transiciona a `IN_REVIEW` cuando el experto entrega avances/renders; puede retornar a `IN_PROGRESS` si el cliente solicita modificaciones, o avanzar a `COMPLETED` si el cliente aprueba el entregable. Asimismo, permite la cancelación a estado `CANCELED` por parte del cliente únicamente desde los estados `PENDING` o `IN_PROGRESS`.

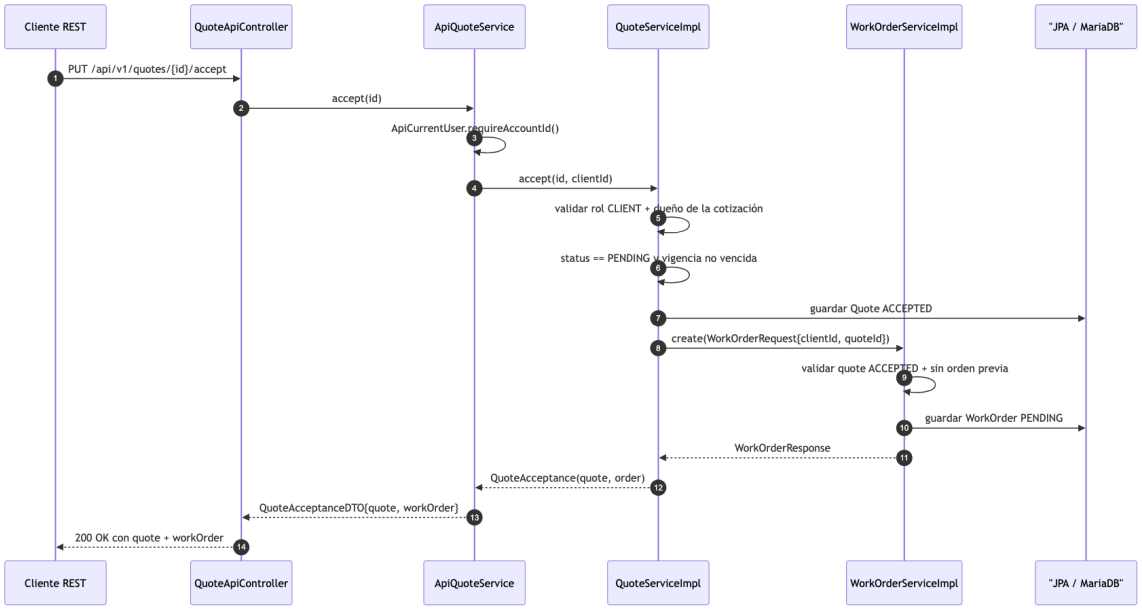


Figura: s07-h-secuencia-cotizacion

- **Flujo REST de Aceptación de Cotización (s07-secuencia-cotizacion):** Muestra el procesamiento de la petición `PUT /api/v1/quotes/{id}/accept` por parte de `QuoteApiController`. A través de `ApiQuoteService` y `ApiCurrentUser`, se obtiene la identidad del usuario autenticado y se delega la ejecución a `QuoteServiceImpl`. Esta capa valida que la cuenta solicitante tenga el rol `CLIENT`, sea la propietaria de la cotización, y que esta se encuentre en estado `PENDING` con fecha de vigencia válida. Al cumplirse estas condiciones, la cotización pasa a estado `ACCEPTED` y se guarda en base de datos. Inmediatamente, `WorkOrderServiceImpl` crea y guarda la orden de trabajo (**`WORK_ORDER`**) vinculada en relación 1:1 con estado inicial `PENDING`, retornando el objeto `QuoteAcceptanceDTO` empaquetado en una respuesta HTTP 200 OK.

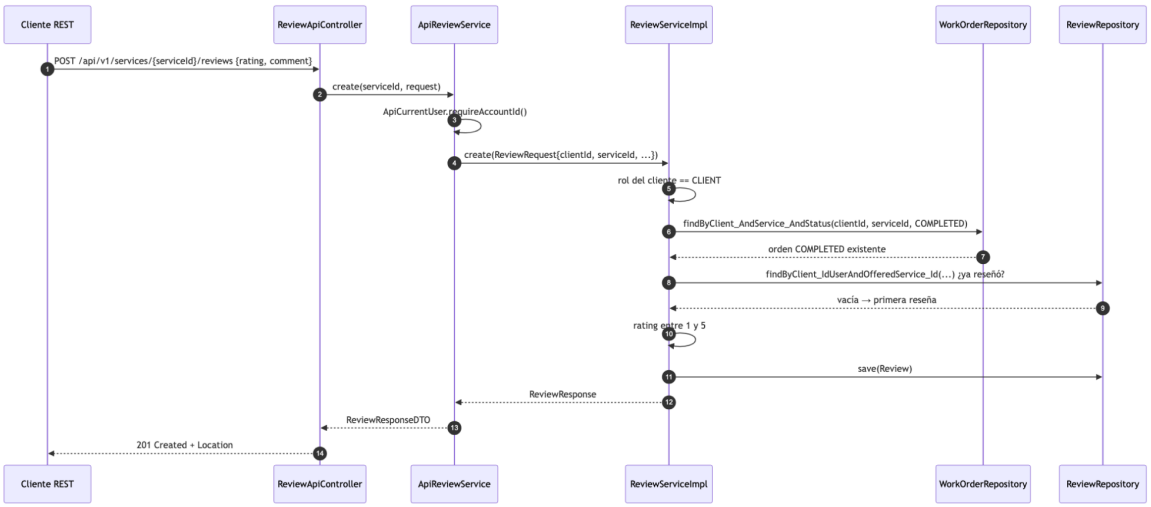


Figura: s07-i-secuencia-resena

- **Flujo REST de Creación de Reseñas (s07-secuencia-resena):** Detalla el flujo desencadenado por `POST /api/v1/services/{serviceId}/reviews` enviado por un cliente. `ReviewApiController` transfiere la solicitud a `ApiReviewService`, el cual obtiene el identificador del usuario autenticado (`ApiCurrentUser`) y llama a `ReviewServiceImpl`. El servicio verifica que el usuario posea el rol `CLIENT` y consulta a `WorkOrderRepository` para confirmar la existencia de al menos una orden de trabajo finalizada (`COMPLETED`) vinculada al cliente y al servicio. Asimismo, mediante `ReviewRepository`, se valida que el cliente no haya registrado una reseña previa para dicho servicio. Finalmente, tras comprobar que la calificación numérica se encuentra en el rango permitido (1 a 5), se persiste la entidad `Review` y se retorna una respuesta HTTP 201 Created con el encabezado `Location` del recurso.

8. Seguridad

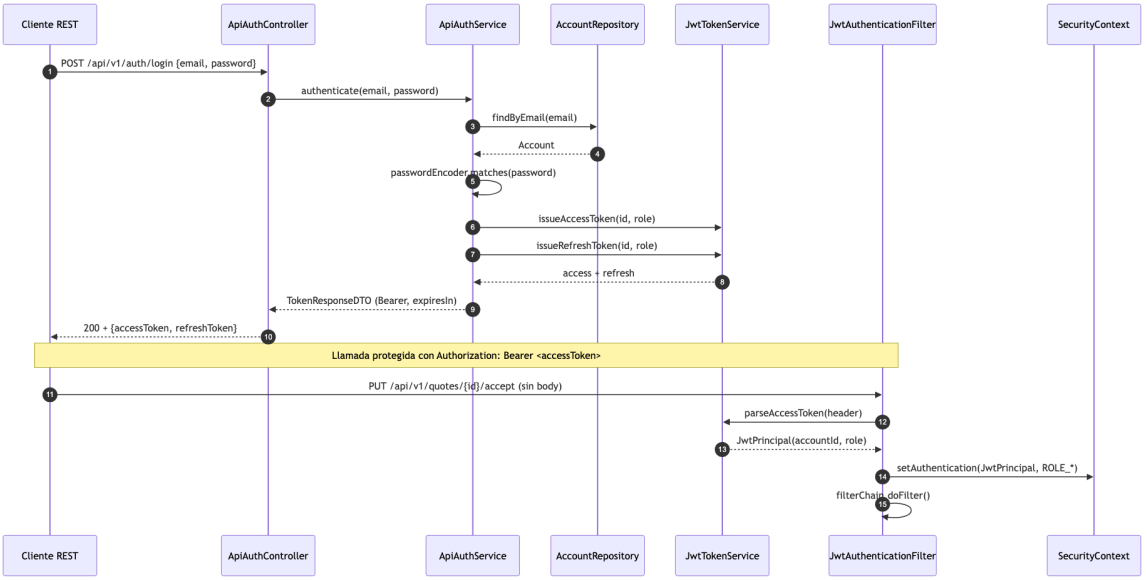


Figura: s08-secuencia-jwt

El diagrama de secuencia de autenticación y JWT (s08-secuencia-jwt) detalla el flujo completo para el control de acceso sin estado en la API REST, involucrando como participantes a **Cliente REST**, **`ApiAuthController`**, **`ApiAuthService`**, **`AccountRepository`**, **`JwtTokenService`**, **`JwtAuthenticationFilter`** y el **`SecurityContext`**:

- **Flujo de Autenticación Inicial (`POST /api/v1/auth/login`):** El Cliente REST envía sus credenciales (`email` y `password`). `ApiAuthController` delega la validación a `ApiAuthService`, el cual consulta la cuenta en `AccountRepository` y verifica el hash de la contraseña mediante `PasswordEncoder`. Una vez autenticado con éxito, se solicitan a `JwtTokenService` los tokens firmados de acceso (`accessToken`) y de refresco (`refreshToken`), devolviendo al cliente una respuesta HTTP 200 con el objeto `TokenResponseDTO`.
- **Flujo de Peticiones Protegidas:** En llamadas posteriores a recursos protegidos (por ejemplo, `PUT /api/v1/quotes/{id}/accept`), la petición incluye el encabezado `Authorization: Bearer <accessToken>`. El filtro `JwtAuthenticationFilter` intercepta la solicitud, utiliza `JwtTokenService` para parsear y validar la firma del token, extrae los datos de identidad empaquetados en `JwtPrincipal` (`accountId` y `role`), y puebla la autenticación dentro del `SecurityContext` de Spring Security antes de permitir el paso a la cadena de filtros.
- **Derivación Segura de la Identidad:** Este mecanismo garantiza que la identidad del usuario y sus permisos asociados se derivan de forma segura del token verificado criptográficamente en cada petición, evitando confiar en parámetros o identificadores enviados de manera explícita en el cuerpo de la solicitud.

9. API REST

9.1 Documentación de Endpoints

SISPRO3D expone una API REST que cubre la totalidad de sus recursos de negocio bajo el prefijo /api/v1. La API consta de 12 recursos y 62 endpoints, todos documentados y cubiertos por pruebas de integración. Es una API sin estado (stateless) que se autentica exclusivamente mediante tokens JWT (Authorization: Bearer <accessToken>), por lo que la identidad del actor —clientId, expertId o adminId— se deriva siempre del token verificado y nunca se envía en el cuerpo de la petición. La única superficie pública es /api/v1/auth/**. Al autenticarse por token, esta capa está exenta de protección CSRF, mientras que la capa web (Thymeleaf) conserva su formulario con CSRF.

9.1 Convenciones generales de la API

Las especificaciones globales de interacción y el comportamiento predeterminado para los servicios web de la plataforma se estructuran a continuación:

Aspecto	Descripción
URL base	`http://localhost:8080/api/v1`
Formato de intercambio	JSON (`Content-Type: application/json`)
Autenticación	Encabezado `Authorization: Bearer <accessToken>` obligatorio para todos los endpoints, exceptuando la ruta pública `/api/v1/auth/**`.
Identidad del actor	Obtenida directamente a partir de la firma del token JWT verificado, omitiendo la lectura de parámetros en el cuerpo o la URL.
Manejo de estado	Arquitectura completamente sin estado (stateless) sin sesiones activas en el servidor.
Protección CSRF	Exención completa de validaciones CSRF en toda la capa API.

Códigos de respuesta HTTP estandarizados

Código	Significado
`200 OK`	Procesamiento exitoso de peticiones de lectura o actualización de datos.

Código	Significado
`201 Created`	Registro exitoso de un nuevo recurso; incluye el encabezado `Location` con la URI correspondiente.
`204 No Content`	Eliminación completada satisfactoriamente sin retornar contenido en el cuerpo de la respuesta.
`400 Bad Request`	Estructura JSON malformada, tipos de datos incompatibles o fallos en las validaciones de entrada.
`401 Unauthorized`	Ausencia, expiración o invalidez de la firma del token de autenticación.
`403 Forbidden`	Privilegios insuficientes por restricciones de rol, falta de propiedad o falta de participación en el recurso.
`404 Not Found`	El recurso solicitado no fue localizado en la base de datos.
`409 Conflict`	Inconsistencia de negocio o violación de restricciones únicas (ej. duplicados o estados incompatibles).
`500 Internal Server Error`	Excepción no controlada en la ejecución del servidor backend.

9.1.1 Módulo de Autenticación (Ruta pública: /api/v1/auth)

Método	Ruta	Descripción
`POST`	`/auth/login`	Autentica las credenciales y genera el par de tokens (`accessToken` + `refreshToken`).
`POST`	`/auth/refresh`	Emite un nuevo token de acceso mediante la presentación de un refresh token válido.
`POST`	`/auth/logout`	Invalida y revoca el token de refresco provisto finalizando la sesión.

Ejemplo de petición de inicio de sesión (`POST /api/v1/auth/login` → HTTP 200 OK):

```JSON
{
  "email": "mariana.stein@estudio.com",
```



```
  "password": "Client123!"
}
```

Respuesta estructurada:

JSON

```
{
  "accessToken": "eyJhbGciOi...",
  "refreshToken": "eyJhbGciOi...",
  "tokenType": "Bearer",
  "expiresIn": 1800,
  "claims": {
    "role": "CLIENT",
    "sub": "5",
    "iss": "sispro3d"
  }
}
```

Por motivos de seguridad y para impedir la enumeración de cuentas, las respuestas de error en la autenticación son genéricas (retornando la misma respuesta HTTP 401 tanto para correos no registrados como para contraseñas incorrectas).

9.1.2 Módulo de Cuentas (/api/v1/accounts)

Método	Ruta	Descripción
GET	`/accounts/{id}`	Obtiene los detalles de una cuenta de usuario específica.
POST	`/accounts`	Registra un nuevo usuario en la plataforma.
PUT	`/accounts/{id}`	Actualiza los datos del perfil de una cuenta.
DELETE	`/accounts/{id}`	Realiza la baja de una cuenta en el sistema.

9.1.3 Módulo de Categorías (/api/v1/categories)

Método	Ruta	Descripción
GET	`/categories`	Obtiene el listado completo de categorías disponibles.
GET	`/categories/{id}`	Consulta la información detallada de una categoría.
POST	`/categories`	Registra una nueva categoría en el catálogo.
PUT	`/categories/{id}`	Modifica la información de una categoría existente.
DELETE	`/categories/{id}`	Elimina una categoría del sistema.

9.1.4 Módulo de Servicios Ofrecidos (/api/v1/services)

Método	Ruta	Descripción
GET	`/services?categoryId=&expertId=`	Consulta la lista de servicios aplicando filtros opcionales.
GET	`/services/{id}`	Obtiene la información detallada de un servicio de modelado 3D.
POST	`/services`	Registra un nuevo servicio en el catálogo (restringido a usuarios con rol `EXPERT`).
PUT	`/services/{id}`	Actualiza los datos de un servicio previamente registrado.
DELETE	`/services/{id}`	Remueve un servicio ofertado del sistema.

9.1.5 Módulo de Cotizaciones (/api/v1/quotes)

Método	Ruta	Descripción
POST	`/quotes`	Genera una solicitud de cotización en estado `PENDING` (exclusivo para rol `CLIENT` en servicios en estado `APPROVED`).

Método	Ruta	Descripción
GET	`/quotes`	Recupera las cotizaciones asociadas al usuario autenticado.
GET	`/quotes/status/{status}`	Filtra las cotizaciones por su estado actual.
GET	`/quotes/service/{serviceId}`	Consulta las cotizaciones vinculadas a un servicio particular.
GET	`/quotes/{id}`	Obtiene los detalles de una cotización específica.
PUT	`/quotes/{id}/reply`	Envía la propuesta técnica con el monto estimado y vigencia (exclusivo para el experto propietario).
PUT	`/quotes/{id}/accept`	Acepta la propuesta y dispara la creación automática de la orden de trabajo (exclusivo para el cliente solicitante).
PUT	`/quotes/{id}/reject`	Declina la propuesta presentada (exclusivo para el cliente solicitante).
PUT	`/quotes/{id}/expire`	Actualiza el estado a `EXPIRED` tras superarse la fecha límite de vigencia.
DELETE	`/quotes/{id}`	Elimina un registro de cotización.

9.1.6 Módulo de Órdenes de Trabajo (/api/v1/work-orders)

Método	Ruta	Descripción
POST	`/work-orders`	Inicializa la orden derivada de una cotización aprobada (devuelve conflicto HTTP 409 si la orden ya existe).
GET	`/work-orders`	Lista las órdenes pertenecientes al usuario autenticado.

Método	Ruta	Descripción
GET	/work-orders/client/{clientId}	Obtiene las órdenes asociadas a un cliente (restringido al propio cliente).
GET	/work-orders/expert/{expertId}	Obtiene las órdenes vinculadas a un experto (restringido al propio experto).
GET	/work-orders/status/{status}	Filtra el listado de órdenes según su estado actual.
GET	/work-orders/{id}	Consulta los detalles de una orden (restringido a las dos partes involucradas).
PUT	/work-orders/{id}/status	Ejecuta la transición de estado verificando las reglas de negocio y los roles autorizados.

9.1.7 Módulo de Entregables (/api/v1/deliverables)

Método	Ruta	Descripción
GET	/work-orders/{orderId}/deliverables	Consulta la lista de archivos entregables vinculados a una orden.
POST	/work-orders/{orderId}/deliverables	Sube y vincula un archivo ejecutable/fuente a la orden activa (exclusivo para el experto asignado).
GET	/deliverables/{id}	Obtiene los metadatos de un entregable (restringido a las partes involucradas).
PUT	/deliverables/{id}	Actualiza los atributos o la descripción del entregable.
DELETE	/deliverables/{id}	Remueve un entregable junto a sus vistas previas asociadas.

9.1.8 Módulo de Vistas Previas y Renders (/api/v1/previews)

Método	Ruta	Descripción
POST	/deliverables/{deliverableId}/previews	Anexa una imagen de renderizado de previsualización (exclusivo para el experto propietario).
GET	/deliverables/{deliverableId}/previews	Lista las previsualizaciones asociadas al entregable en orden cronológico.
GET	/previews/{id}	Consulta los datos de una previsualización individual.
PUT	/previews/{id}	Actualiza una imagen de previsualización previamente cargada.
DELETE	/previews/{id}	Elimina la vista previa seleccionada.

9.1.9 Módulo de Hilos de Conversación (/api/v1/threads)

Método	Ruta	Descripción
POST	/work-orders/{orderId}/thread	Inicializa el chat de la orden (retorna conflicto HTTP 409 si la orden ya cuenta con un hilo).
GET	/work-orders/{orderId}/thread	Recupera la información del hilo de comunicación de una orden.
GET	/threads/{id}	Obtiene los datos de un hilo mediante su identificador único.
DELETE	/threads/{id}	Elimina el hilo especificado (restringido a las partes del proyecto).

9.1.10 Módulo de Mensajería (/api/v1/messages)

Método	Ruta	Descripción
POST	/threads/{threadId}/messages	Publica una interacción dentro del hilo (restringido a participantes y órdenes activas).

Método	Ruta	Descripción
GET	/threads/{threadId}/messages	Lista los mensajes enviados dentro del hilo ordenados por fecha de creación.
GET	/messages/{id}	Consulta el detalle de un mensaje individual.
DELETE	/messages/{id}	Remueve un mensaje específico.

9.1.11 Módulo de Reseñas y Evaluaciones (/api/v1/reviews)

Método	Ruta	Descripción
GET	/services/{serviceId}/reviews	Recupera las evaluaciones publicadas para un servicio específico.
POST	/services/{serviceId}/reviews	Registra una reseña (restringido a usuarios con rol CLIENT y órdenes en estado COMPLETED).
PUT	/reviews/{id}	Edita la calificación o comentario de una reseña existente.
DELETE	/reviews/{id}	Elimina una reseña del sistema.

9.1.12 Módulo de Servicios Favoritos (/api/v1/.../favorite-services)

Método	Ruta	Descripción
POST	/clients/{clientId}/favorite-services/{serviceId}	Agrega un servicio a favoritos (devuelve HTTP 403 si el id no corresponde al usuario autenticado).
GET	/clients/{clientId}/favorite-services	Lista los servicios marcados como favoritos por un cliente.
DELETE	/clients/{clientId}/favorite-services/{serviceId}	Remueve un servicio de la lista de favoritos.
POST	/me/favorite-services/{serviceId}	Agrega un servicio a favoritos derivando el id directamente del token del cliente.

Método	Ruta	Descripción
`GET`	`/me/favorite-services`	Lista los servicios marcados como favoritos por el cliente autenticado.
`DELETE`	`/me/favorite-services/{serviceId}`	Remueve un servicio de favoritos derivando el id directamente del token del cliente.

Estructura unificada de respuestas de error (ErrorDetail)

Las excepciones interceptadas centralizadamente por el gestor global devuelven un cuerpo de respuesta estandarizado según la siguiente estructura:

```JSON
{
  "timestamp": "2026-10-14T20:30:00",
  "status": 403,
  "error": "Forbidden",
  "message": "Only the expert owning the order can perform this action",
  "path": "/api/v1/work-orders/3/deliverables",
  "details": []
}
```

Mapeo de códigos de error según el escenario de excepción

Escenario o regla gatillada	Código HTTP devuelto
Errores de validación en DTO, JSON malformado o incompatibilidad de tipos.	`400 Bad Request`
Ausencia de token o firma de autenticación inválida/expirada.	`401 Unauthorized`
Restricciones por jerarquía de rol, falta de propiedad o falta de participación en la orden.	`403 Forbidden`
Recurso solicitado inexistente en la base de datos.	`404 Not Found`
Registros duplicados o violaciones de integridad relacional.	`409 Conflict`
Cualquier otra excepción interna no controlada explícitamente.	`500 Internal Server Error`

Flujo de prueba e integración mediante cURL

1. Autenticación y obtención del token de acceso:

```shell
TOKEN=$(curl -s -X POST http://localhost:8080/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d
'{"email":"mariana.stein@estudio.com","password":"Client123!"}' |
jq -r .accessToken)
```

2. Registro de solicitud de cotización:

```shell
curl -s -X POST http://localhost:8080/api/v1/quotes \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type:
application/json" \
  -d '{"offeredServiceId":2,"description":"Sculpt realista"}'
```

3. Aceptación de la propuesta por parte del cliente (generando la orden de trabajo):

```shell
curl -s -X PUT http://localhost:8080/api/v1/quotes/1/accept -H
"Authorization: Bearer $TOKEN"
```

4. Consulta de la orden de trabajo creada:

```shell
curl -s http://localhost:8080/api/v1/work-orders/1 -H
"Authorization: Bearer $TOKEN"
```

La especificación detallada de los 62 endpoints se encuentra documentada en la ruta `docs/api-rest-endpoints.md`, complementada con la colección ejecutable disponible en `docs/postman_collection.json`.

9.2 Integración con Servicios Web Externos

A fin de enriquecer la experiencia de usuario y proveer montos de referencia en divisas internacionales, la aplicación consume un servicio externo de conversión de moneda para presentar, junto al precio original en Pesos Mexicanos (MXN), su equivalente calculado en Dólares Estadounidenses (USD).

Aspecto	Detalle técnico
Proveedor de API	Frankfurter (`https://api.frankfurter.dev`)
Endpoint consumido	`GET /v1/latest?base=MXN&symbols=USD`
Dirección de conversión	MXN → USD
Estrategia de consumo	Consumo exclusivamente del lado del servidor (el cliente/navegador no realiza llamadas directas al proveedor externo).
Componentes receptores	Vistas del catálogo (`/services`) y detalle del servicio (`/services/{id}`).
Cliente HTTP utilizado	`RestClient` de Spring 6 con configuraciones explícitas de tiempo de espera.

Estructura y responsabilidad de componentes de la integración

Componente	Responsabilidad
`ExchangeRateProperties`	Mapea las propiedades de configuración externas (`external.exchange-rate.*`) como estado, URL, divisas y tiempos límite.
`ExchangeRateConfig`	Construye y expone el bean `RestClient` estableciendo tiempos de espera de conexión y lectura en 2 segundos.
`ExchangeRateClient`	Interfaz de contrato para la obtención del tipo de cambio.
`FrankfurterExchangeRateClient`	Implementación concreta que ejecuta la llamada a Frankfurter y transforma el nodo `rates.USD` a un objeto `BigDecimal`.
`ExchangeRateService`	Servicio de orquestación que gestiona la conversión de moneda, el almacenamiento en caché diario y la estrategia de degradación de servicio.

Configuración en application.properties

```
None
external.exchange-rate.enabled=true
external.exchange-rate.base-url=https://api.frankfurter.dev
external.exchange-rate.base-currency=MXN
```

Axel Fernando Montiel Aviles
Universidad Nacional Autónoma de México



```
external.exchange-rate.target-currency=USD
external.exchange-rate.connect-timeout=2s
external.exchange-rate.read-timeout=2s
```

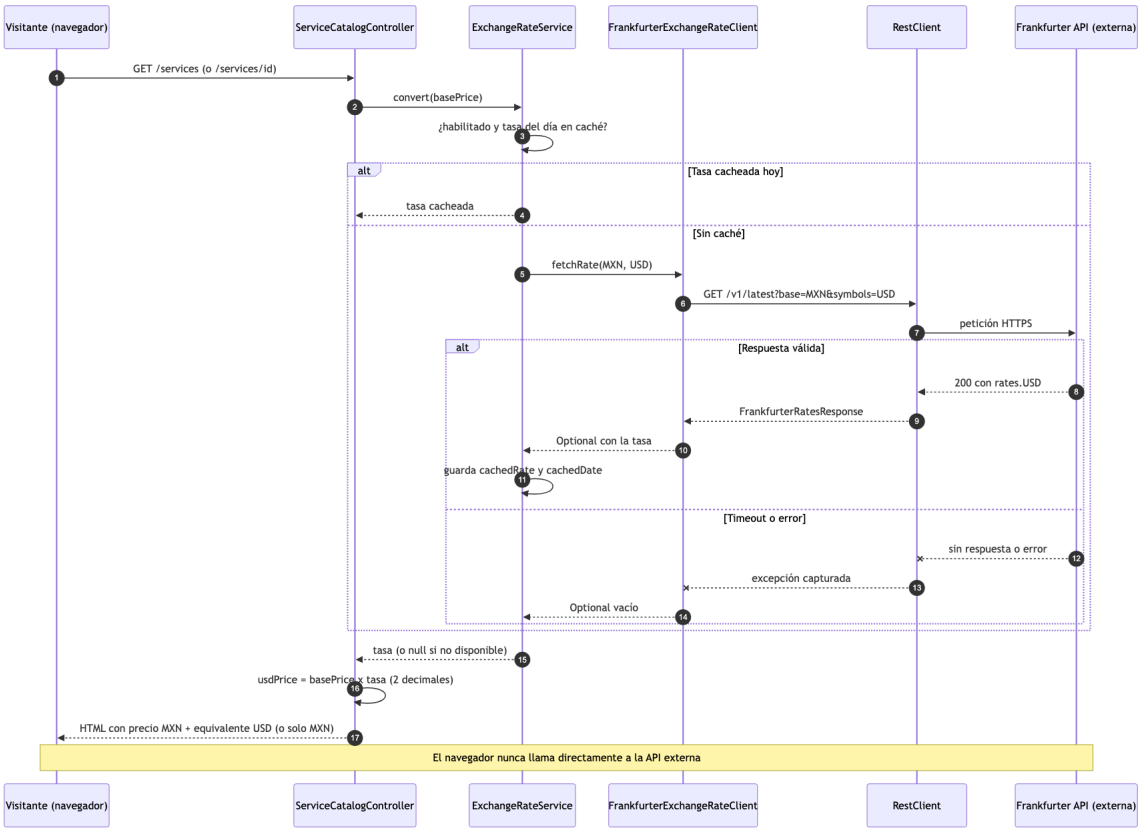
Tolerancia a fallos y mecanismos de resiliencia

- **Límites de tiempo estricto:** Tiempos de espera para conexión y lectura fijados en 2 segundos para no degradar el tiempo de respuesta de las páginas web.
- **Caché en memoria:** Conserva el valor obtenido (`cachedRate` y `cachedDate`) realizando una única llamada externa por día.
- **Manejo de degradación de servicio (Graceful Degradation):** Si la API externa no responde, agota el tiempo de espera o retorna una respuesta no válida, el servicio retorna `null` de manera silenciosa, permitiendo que la vista se renderice únicamente con el monto en pesos mexicanos sin detener la aplicación.
- **Integridad del valor base:** El monto en MXN y sus formatos se mantienen como la referencia principal; el equivalente en USD se presenta como información adicional.
- **Procesamiento del lado del servidor:** La conversión es resuelta durante la fase de renderizado en el backend, omitiendo peticiones del navegador hacia la API de terceros.

Flujo de ejecución para la conversión

- 1. El cliente accede a la ruta del catálogo público (`/services` o `/services/{id}`).
- 2. El controlador `ServiceCatalogController` invoca el método `ExchangeRateService.convert(...)`.
- 3. El servicio consulta la tasa almacenada en caché para la fecha actual o la solicita al proveedor externo mediante `RestClient`.
- 4. El controlador asigna el valor convertido a los atributos del modelo (`usdPrices` / `usdPrice`) para su despliegue en la plantilla Thymeleaf.

5. Ante cualquier error de comunicación, el valor en USD es omitido y la página carga normalmente.



10. Configuración del Sistema

La plataforma centraliza sus parámetros operativos en el archivo de propiedades principal `src/main/resources/application.properties` (mediante pares clave-valor). Adicionalmente, se dispone del perfil aislado `src/test/resources/application-test.yml` (en formato YAML) habilitado explícitamente mediante la anotación `@ActiveProfiles("test")`. Con el fin de atender criterios de seguridad y despliegue, las credenciales sensibles —tales como la clave secreta JWT y los accesos a la base de datos— admiten su sobrescritura mediante variables de entorno o parámetros del contenedor de aplicación.

10.1 Aplicación y Motor de Vistas

Propiedad	Valor / Configuración
`spring.application.name`	`sispro3d`
`spring.thymeleaf.cache`	`false`
`spring.messages.basename`	`messages`
`spring.messages.encoding`	`UTF-8`

10.2 Persistencia y Base de Datos (MariaDB)

Propiedad	Valor de producción / desarrollo
`spring.datasource.url`	`jdbc:mariadb://localhost:3306/sispro3d_db?useSSL=false&serverTimezone=UTC`
`spring.datasource.driverClassName`	`org.mariadb.jdbc.Driver`
`spring.datasource.username`	`appuser`
`spring.datasource.password`	`MiPasswordSegura123!`
`spring.datasource.hikari.connection-timeout`	`20000`
`spring.datasource.hikari.maximum-pool-size`	`5`
`spring.jpa.database-platform`	`org.hibernate.dialect.MariaDBDialect`
`spring.jpa.hibernate.ddl-auto`	`update`
`spring.jpa.show-sql`	`true`
`spring.jpa.properties.hibernate.format_sql`	`true`
`spring.sql.init.mode`	`always`
`spring.jpa.defer-datasource-initialization`	`true`

El esquema DDL del sistema se define en `src/main/resources/schema.sql` (diseñado para MariaDB con soporte `utf8mb4` y campos tipo `ENUM`), ejecutándose de manera automatizada al inicializar la aplicación. Para poblar información de prueba, se incluye la secuencia de inserción en `scripts/data.sql`, ejecutable manualmente a través de la CLI del motor SQL.

10.3 Seguridad y Parámetros JWT

Propiedad	Valor por defecto
`app.jwt.secret`	`${SISPRO3D_JWT_SECRET:dev-only-signing-key-cambiar-en-produccion-0123456789}`
`app.jwt.access-ttl`	`PT30M`

Propiedad	Valor por defecto
`app.jwt.refresh-ttl`	`P7D`
`app.jwt.issuer`	`sispro3d`

La clave criptográfica de firma dispone de una representación predeterminada delimitada al entorno de desarrollo. En ambientes de producción, esta debe ser reemplazada mediante la variable `SISPRO3D_JWT_SECRET`. La vigencia de las fichas de acceso utiliza la sintaxis ISO-8601 (30 minutos para tokens de acceso y 7 días para tokens de refresco).

10.4 Almacenamiento de Entregables

Propiedad	Configuración
`app.storage.location`	`uploads`
`spring.servlet.multipart.max-file-size`	`110MB`
`spring.servlet.multipart.max-request-size`	`110MB`

El límite global fijado en la capa de transmisión de Spring (110 MB) se establece con una tolerancia superior respecto al umbral máximo de negocio (100 MB para archivos de modelado 3D), asegurando que la validación sea procesada por el controlador interno y devuelva una respuesta estandarizada al cliente.

10.5 Integración de API Externa (Frankfurter)

Propiedad	Valor asignado
`external.exchange-rate.enabled`	`true`
`external.exchange-rate.base-url`	`https://api.frankfurter.dev`
`external.exchange-rate.base-currency`	`MXN`
`external.exchange-rate.target-currency`	`USD`
`external.exchange-rate.connect-timeout`	`2s`



Propiedad	Valor asignado
`external.exchange-rate.read-timeout`	`2s`

Esta integración opera de forma pública sin requerir llaves privadas. En caso de interrupción en la API externa, el módulo degrada suavemente desplegando los precios únicamente en Pesos Mexicanos. Las propiedades son mapeadas mediante el componente anotado con `@ConfigurationProperties(prefix = "external.exchange-rate")`.

10.6 Ambiente de Pruebas Automatizadas (application-test.yml)

La ejecución de la suite de pruebas automatizadas utiliza un perfil aislado para prescindir de dependencias externas y bases de datos físicas:

```
None
spring:
  datasource:
    url: jdbc:h2:mem:testdb;DB_CLOSE_DELAY=-1;MODE=MySQL
    driver-class-name: org.h2.Driver
    username: sa
    password:
  jpa:
    hibernate:
      ddl-auto: create-drop
  sql:
    init:
      mode: never
app:
  jwt:
    secret:
test-only-signing-key-para-los-tests-de-sispro3d-0123456789
    access-ttl: PT30M
    refresh-ttl: P7D
    issuer: sispro3d-test
  storage:
    location: target/test-uploads
external:
```



```
  exchange-rate:
    base-url: http://127.0.0.1:1
    connect-timeout: 500ms
    read-timeout: 500ms
```

Resumen de comportamiento en pruebas integradas

Componente	Estrategia aplicada
Base de datos	Uso de H2 en memoria con compatibilidad MySQL sin requerir una instancia activa de MariaDB.
Generación de esquema	Construcción dinámica estructurada por Hibernate (`create-drop`).
Datos iniciales	Omisión de scripts SQL (`sql.init.mode=never`) para garantizar independencia en los tests.
Servicios de red	Redirección a endpoint inalcanzable, forzando la degradación local sin solicitudes salientes.

10.7 Mecanismos de Sobrescritura en Despliegue

Para entornos de producción, las propiedades sensibles deben inyectarse mediante variables de entorno del sistema operativo:

Shell

```
export
SISPRO3D_JWT_SECRET="clave-larga-y-aleatoria-de-al-menos-32-bytes
"
export SPRING_DATASOURCE_PASSWORD="otra-password"
mvn spring-boot:run
```

Alternativamente, los parámetros pueden ser especificados como argumentos de inicio al ejecutar el paquete compilado:

Shell

```
mvn spring-boot:run
-Dspring-boot.run.arguments="--app.jwt.issuer=sispro3d-prod
--external.exchange-rate.enabled=false"
```

11. Evidencia de Funcionamiento

11.1 Interfaz de Administración

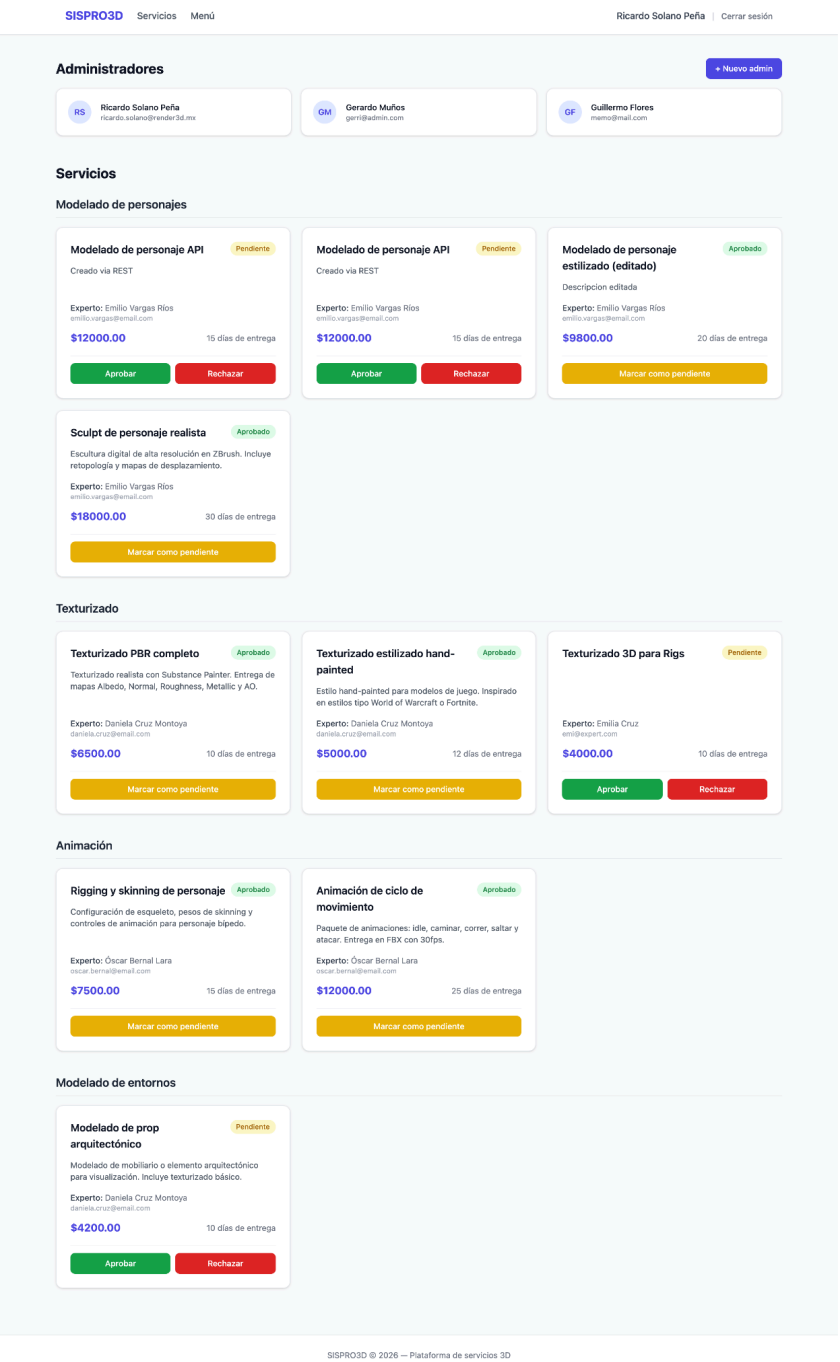


Figura 11.1: Dashboard principal de administración para la gestión global de usuarios y moderación del catálogo de servicios 3D.

Axel Fernando Montiel Aviles
Universidad Nacional Autónoma de México

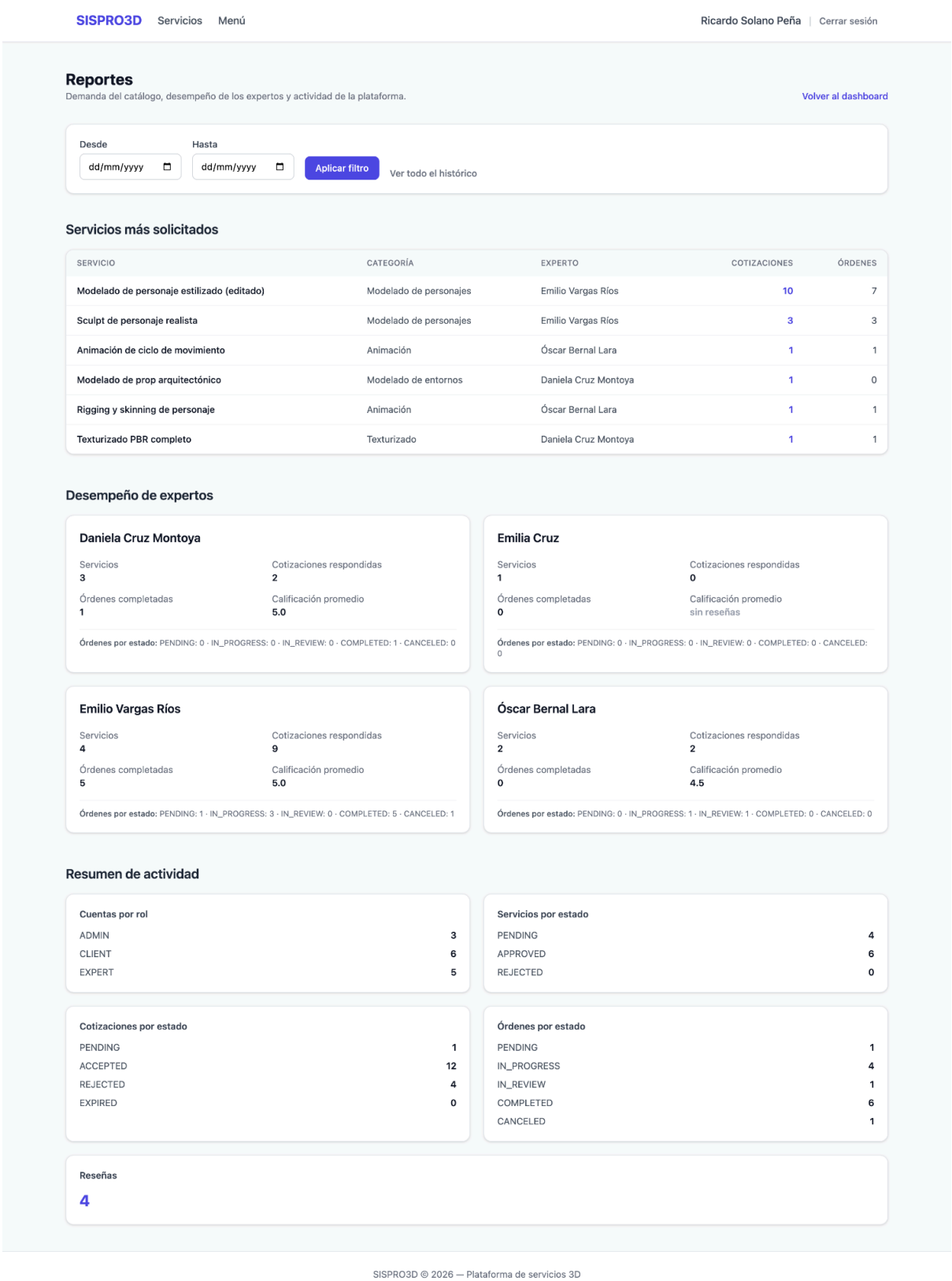


Figura 11.2: Módulo de reportes y analítica avanzada con filtros temporales, métricas de desempeño de expertos y resumen de actividad del sistema.

Axel Fernando Montiel Aviles
Universidad Nacional Autónoma de México

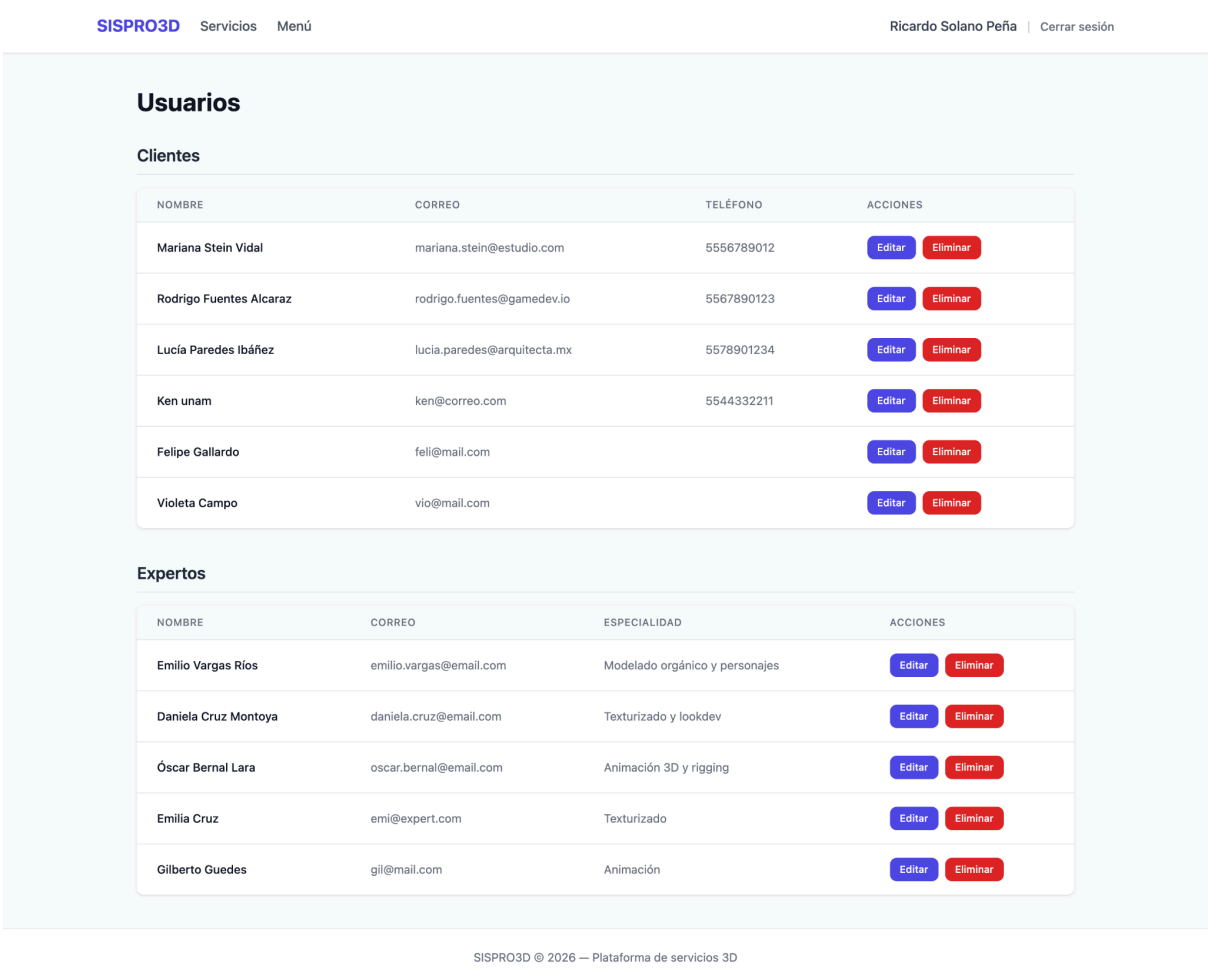


Figura 11.3: Panel de administración de cuentas de usuario clasificado por roles de Cliente y Experto con opciones de edición y eliminación.

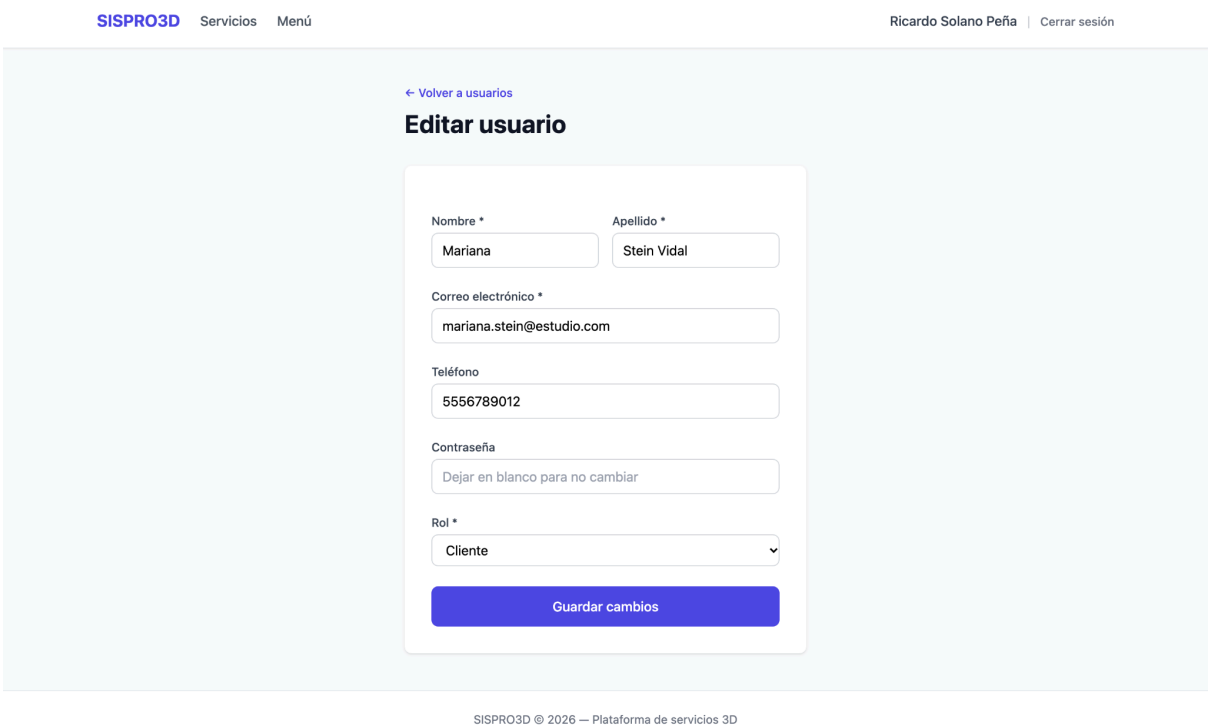


Figura 11.4: Formulario para la modificación de parámetros de perfil y asignación de roles de usuario en la plataforma.

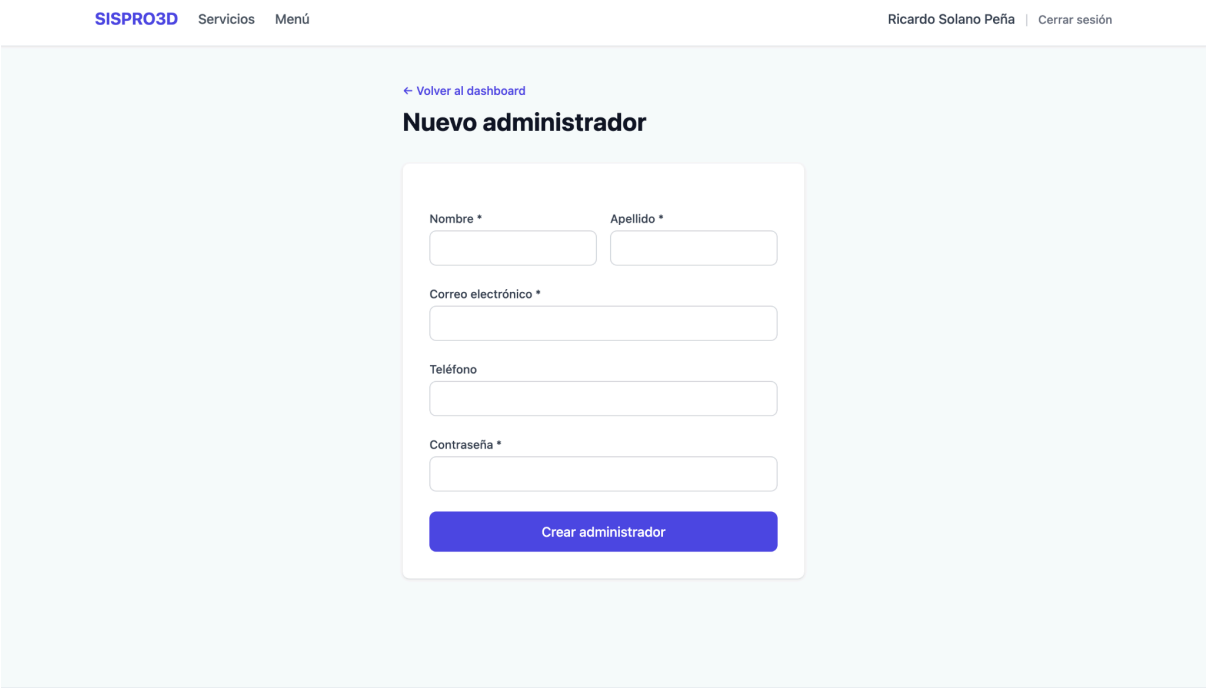


Figura 11.5: Interfaz de registro directo para el alta de nuevos usuarios con privilegios administrativos.

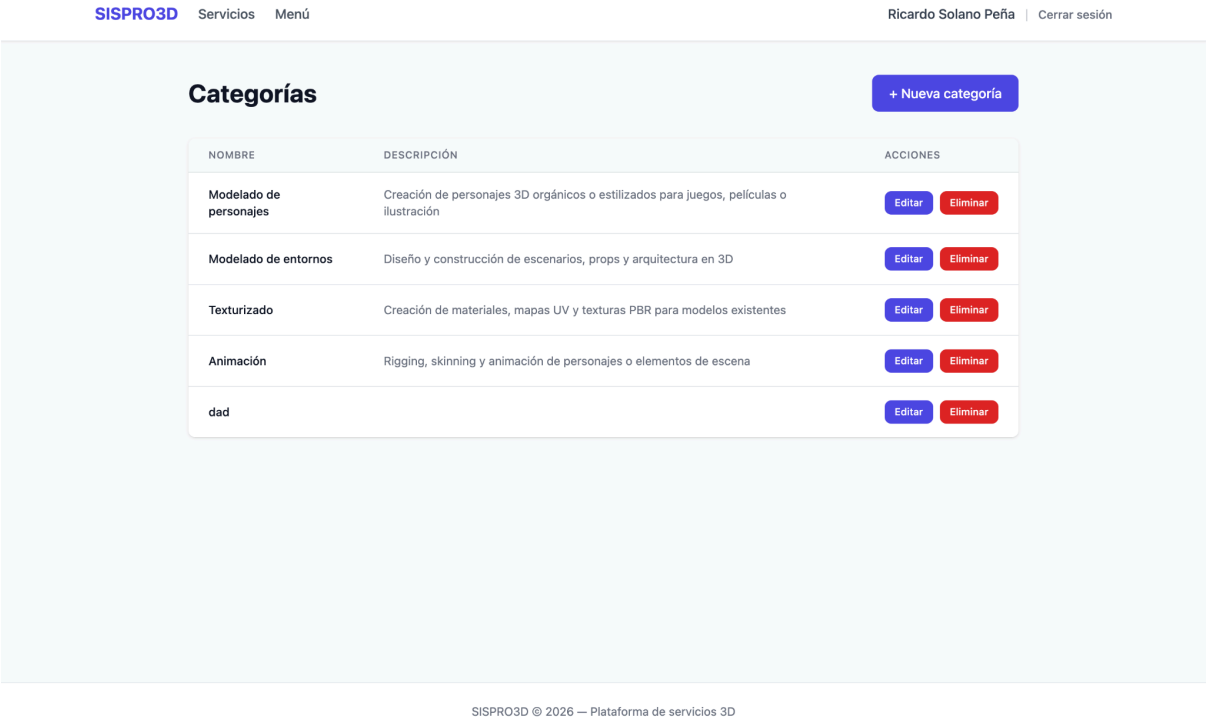


Figura 11.6: Catálogo de gestión de categorías con operaciones CRUD para la organización de los servicios.

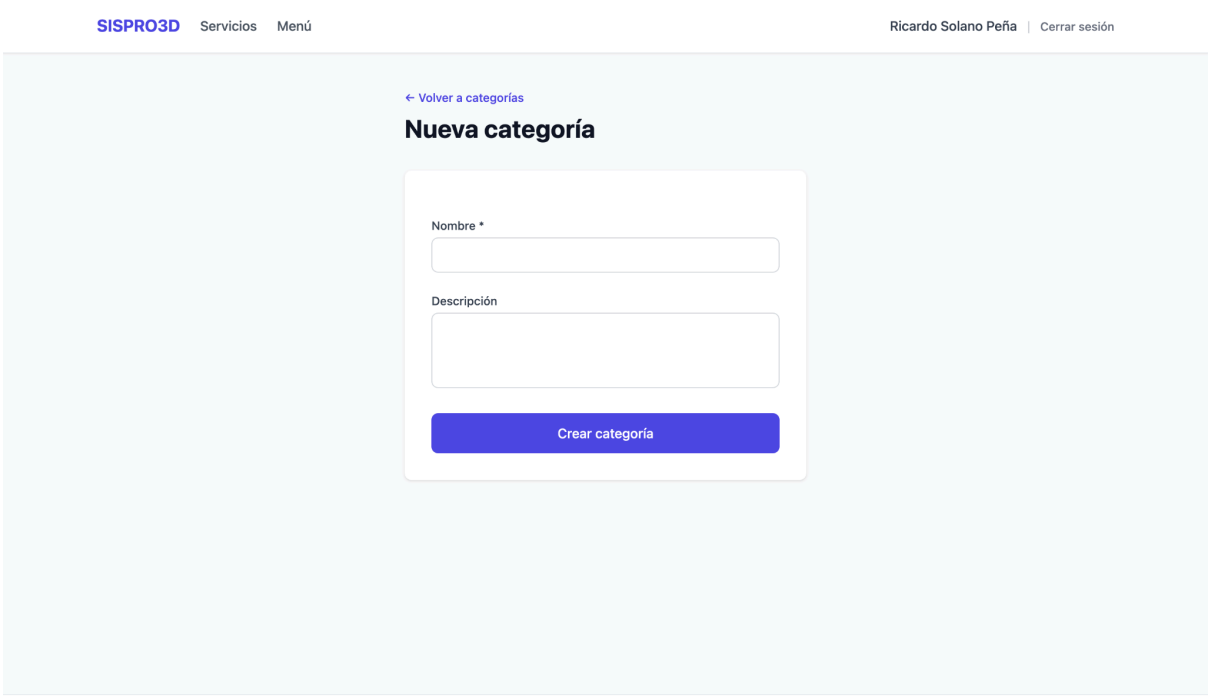


Figura 11.7: Formulario para la creación e incorporación de nuevas categorías en el catálogo del sistema.

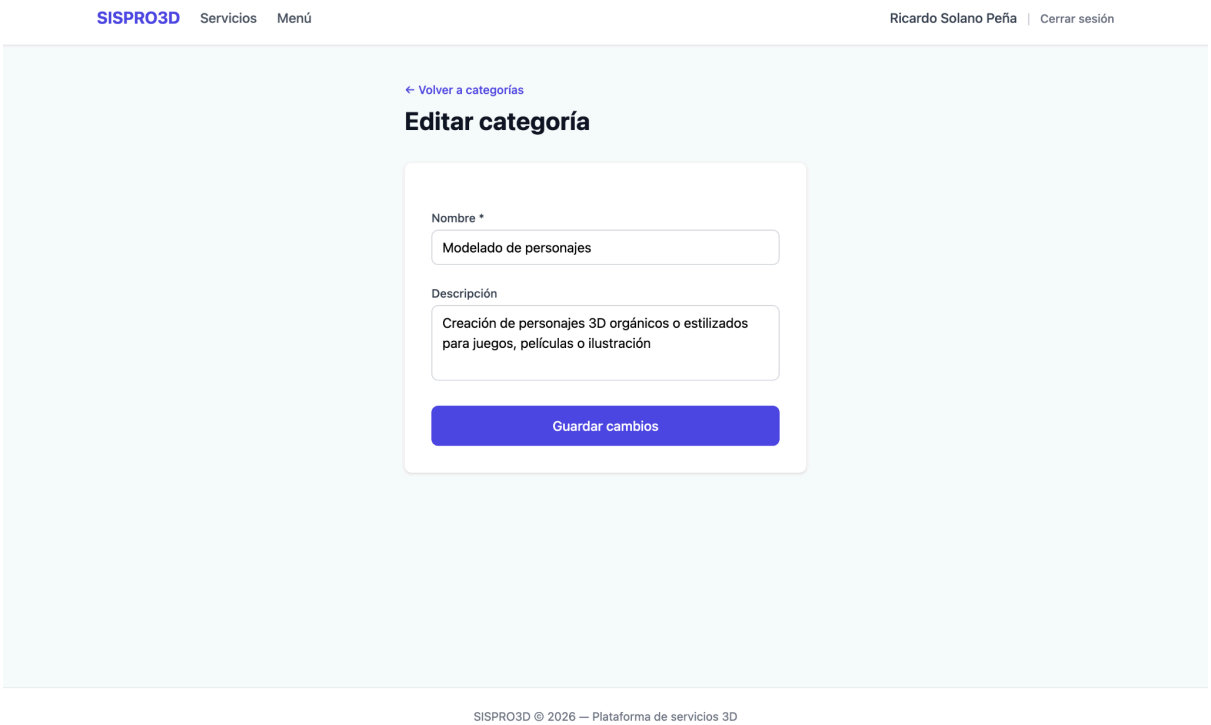


Figura 11.8: Interfaz de edición para la actualización del nombre y descripción de categorías existentes.

11.2 Vistas del Rol Cliente

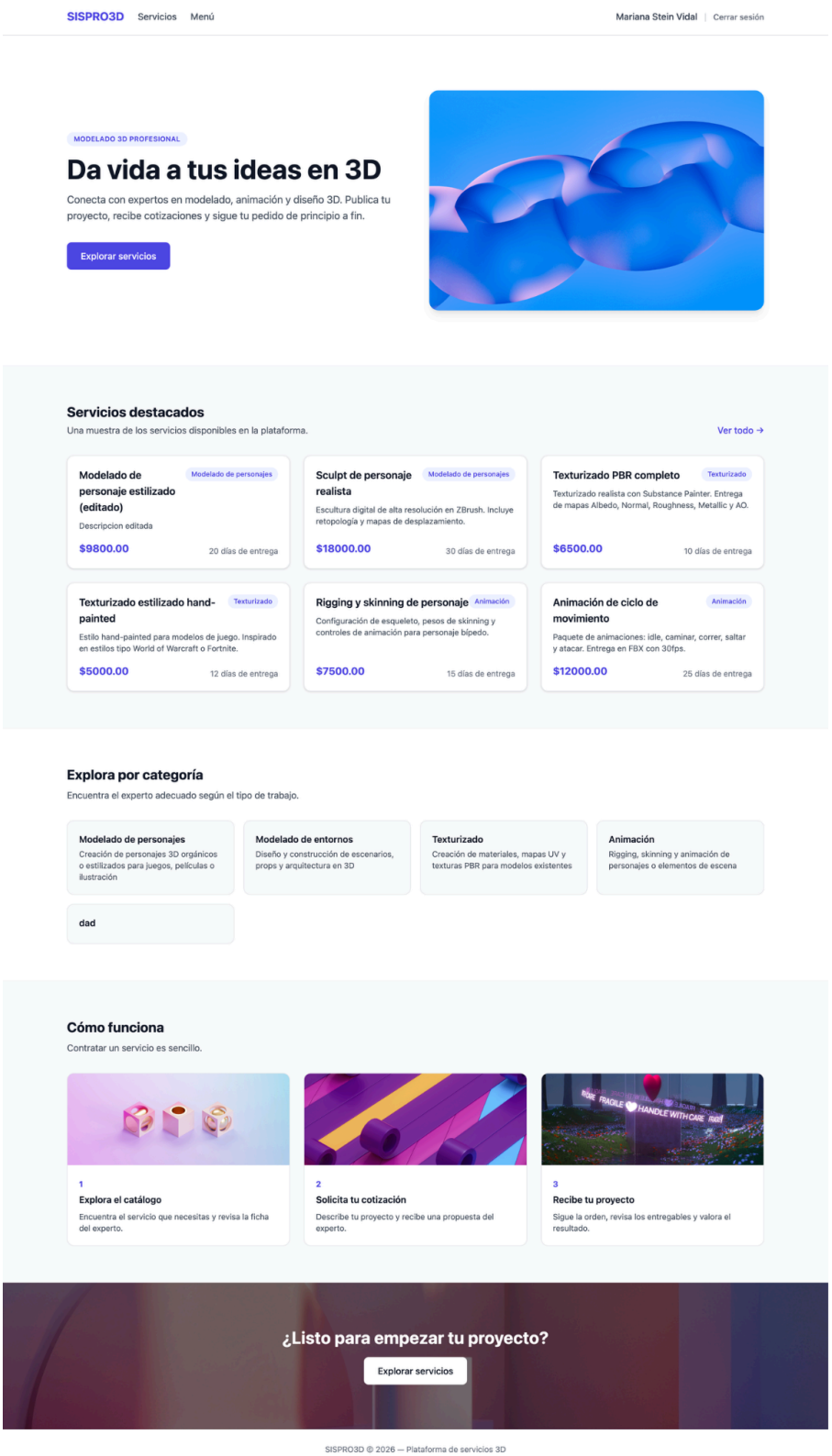


Figura 11.9: Página de inicio para el rol Cliente con servicios destacados, exploración por categoría y flujo de contratación.

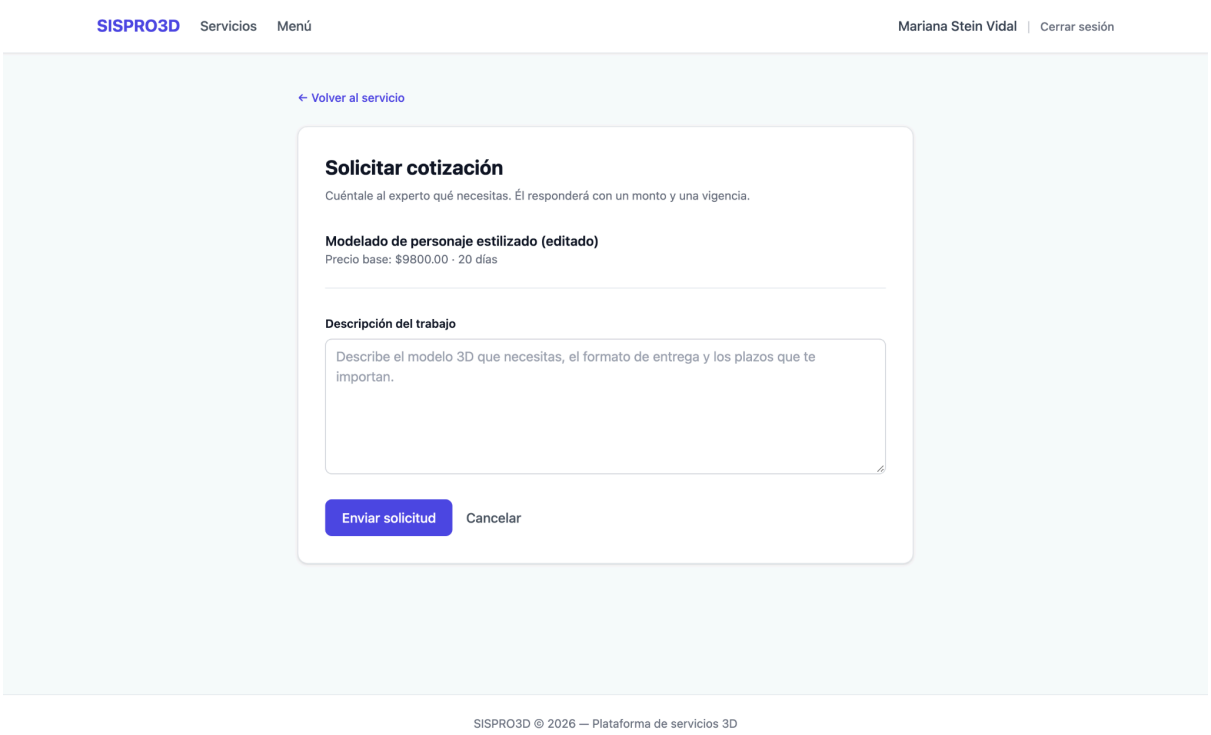


Figura 11.10: Formulario de solicitud de cotización personalizada especificando el alcance del proyecto al experto.

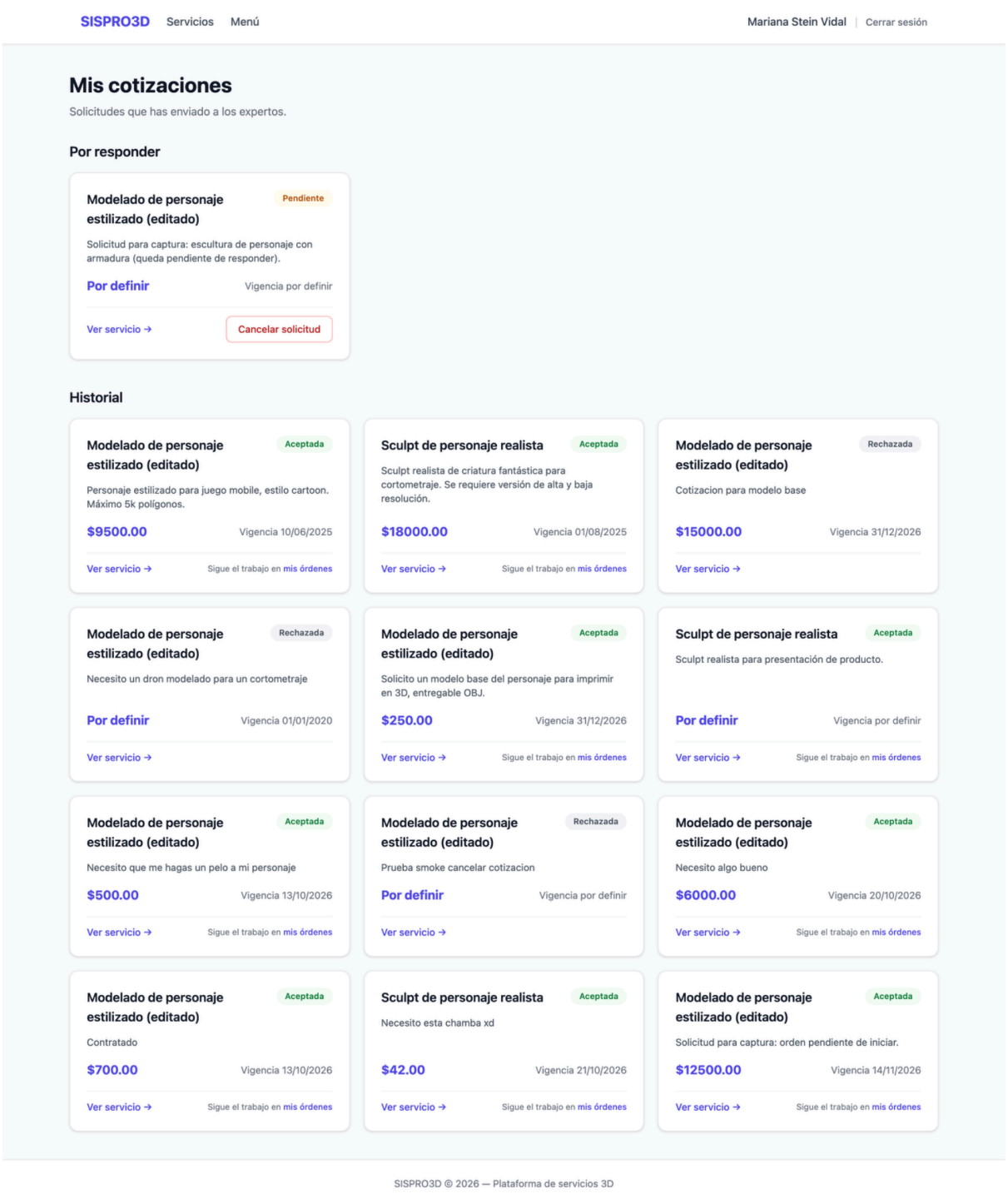


Figura 11.11: Panel de seguimiento de cotizaciones enviadas clasificado por estado actual e historial de interacciones.

Axel Fernando Montiel Aviles
Universidad Nacional Autónoma de México

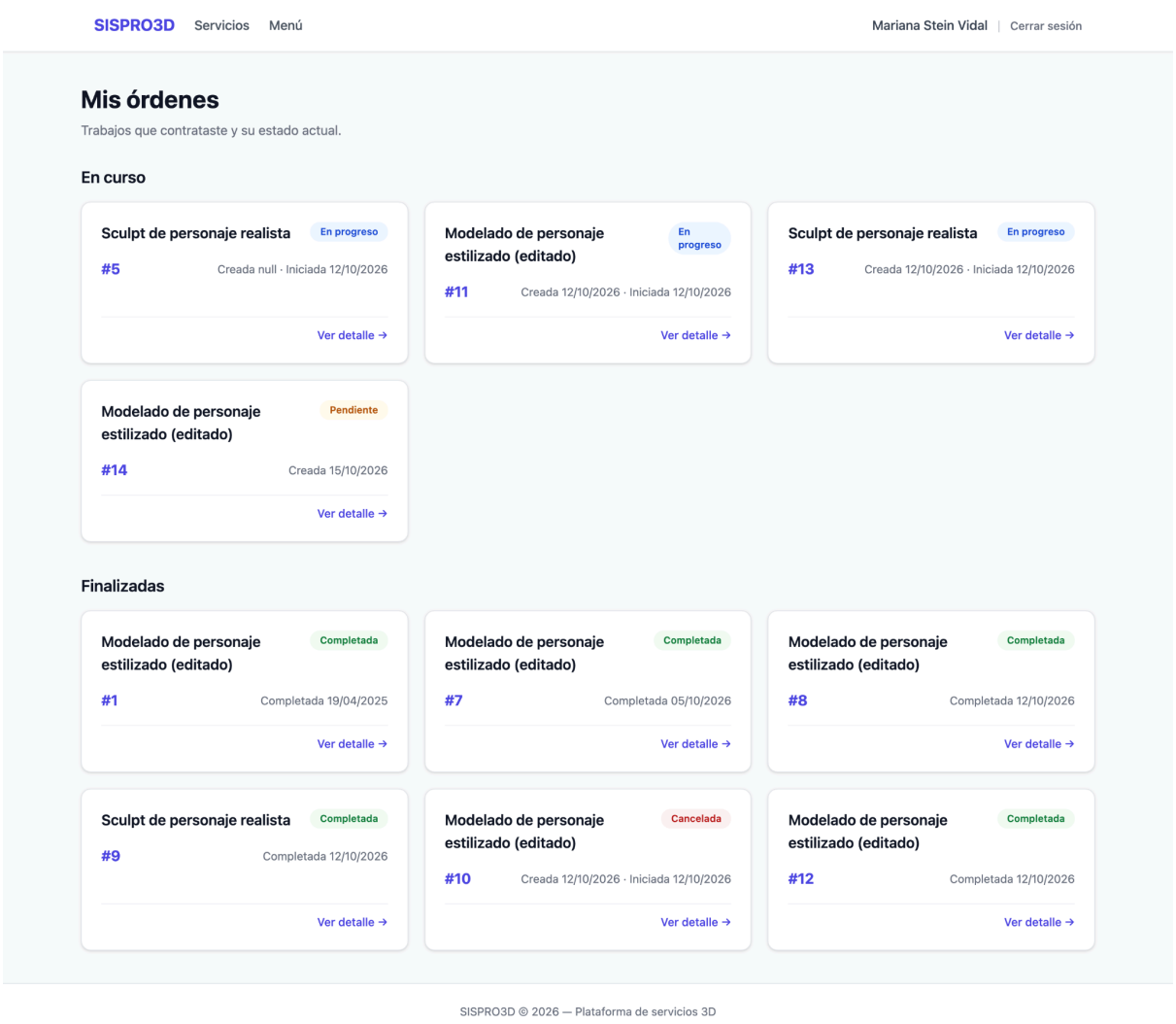


Figura 11.12: Listado de órdenes de servicio del cliente divididas en proyectos en curso y finalizados.

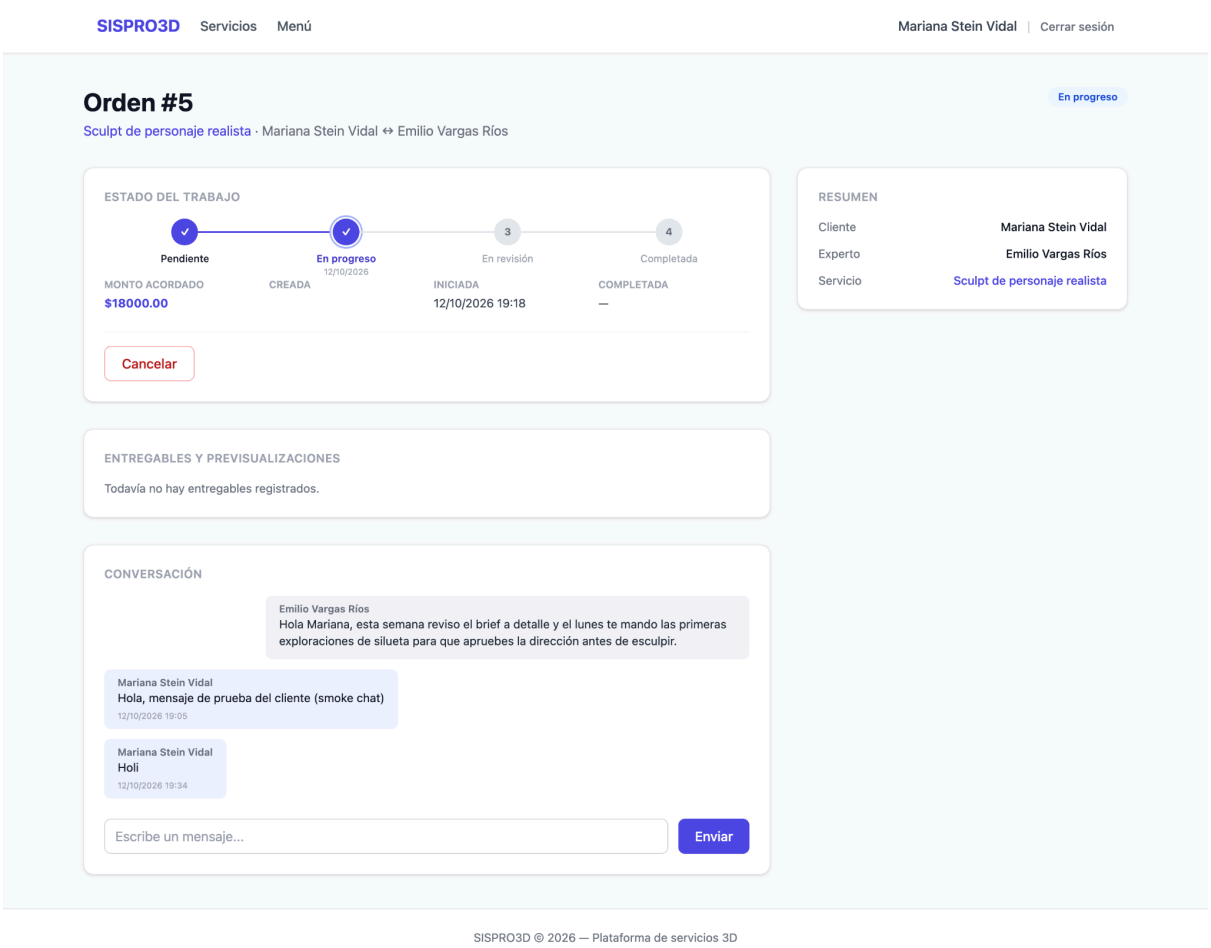


Figura 11.13: Detalle de orden de trabajo activa mostrando el indicador de progreso, entregables e hilo de conversación.

Axel Fernando Montiel Aviles
Universidad Nacional Autónoma de México

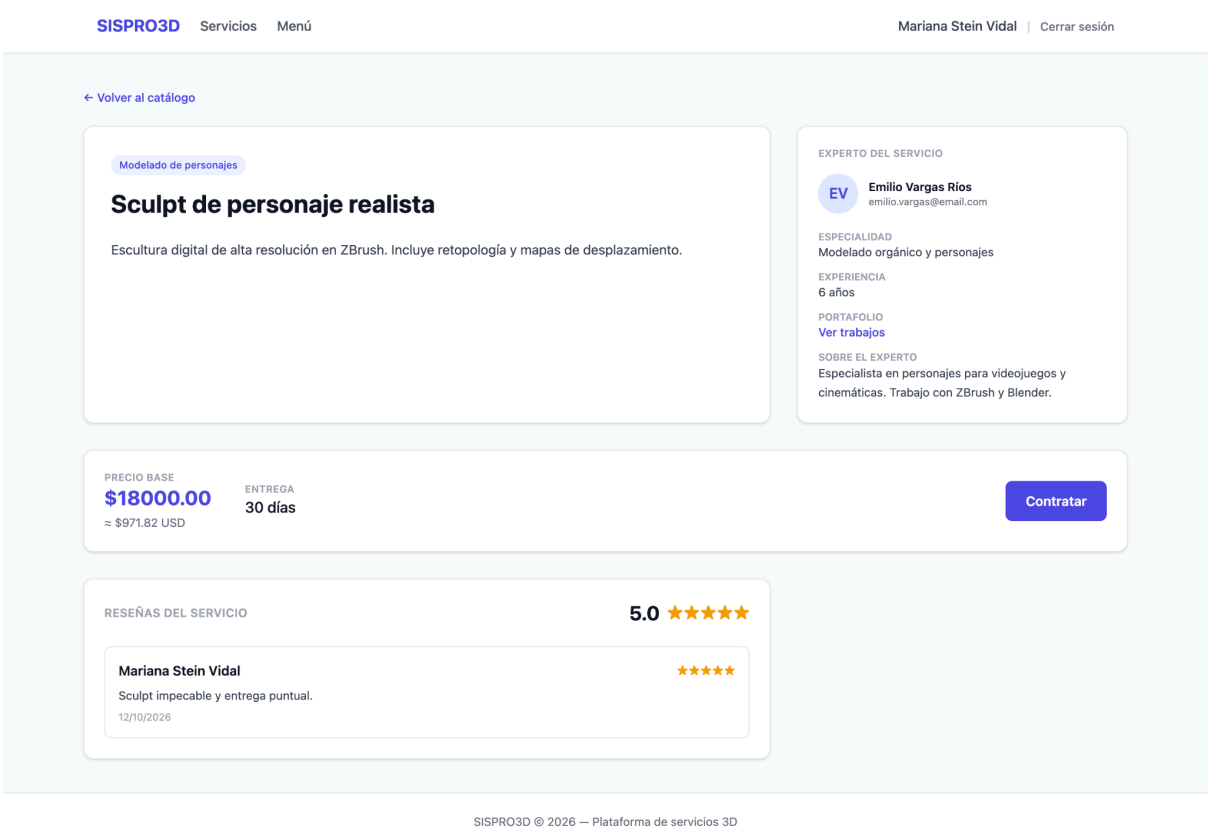


Figura 11.14: Vista detallada de un servicio del catálogo con información del experto, precio base, conversión a USD y reseñas.

Axel Fernando Montiel Aviles
Universidad Nacional Autónoma de México

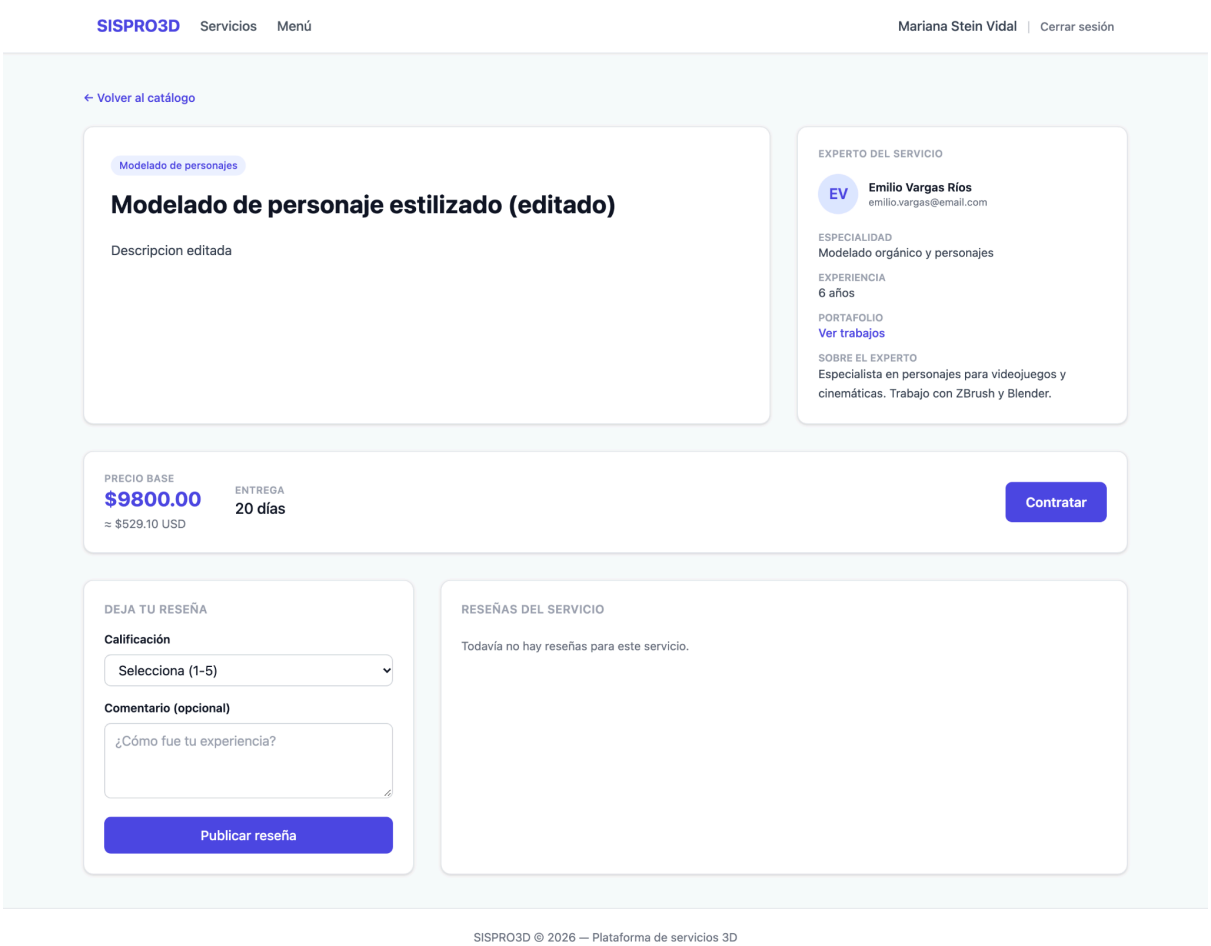


Figura 11.15: Sección de evaluación para el registro de calificación y comentarios tras completar la orden de servicio.

Axel Fernando Montiel Aviles
Universidad Nacional Autónoma de México

11.3 Vistas del Rol Experto

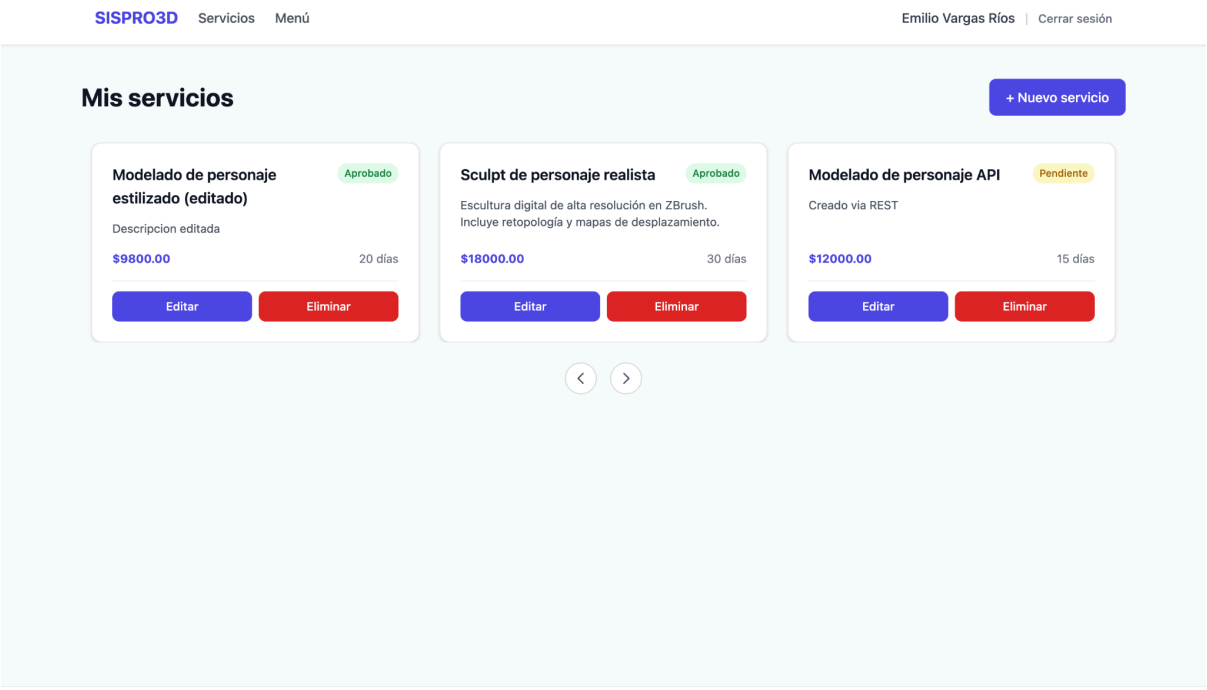


Figura 11.16: Dashboard del experto con la lista de servicios ofertados y sus estados de aprobación por administración.

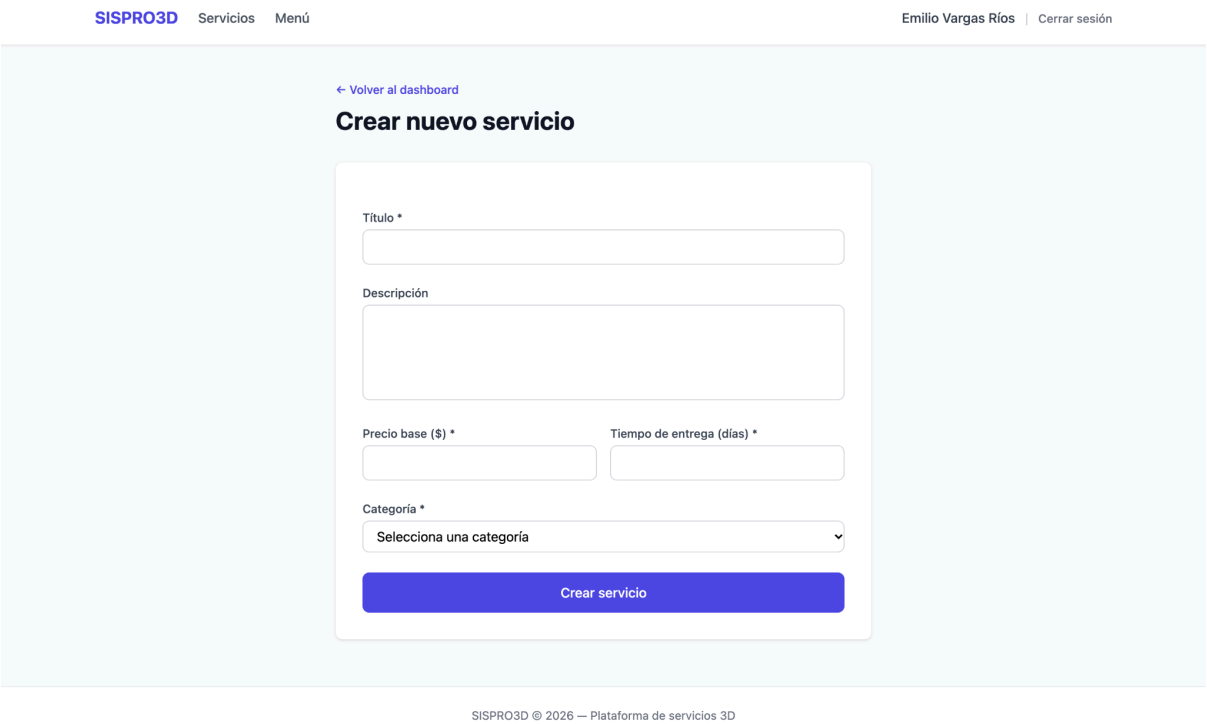


Figura 11.17: Formulario de creación de un nuevo servicio de modelado 3D definiendo título, precio base y plazo de entrega.

Axel Fernando Montiel Aviles
Universidad Nacional Autónoma de México

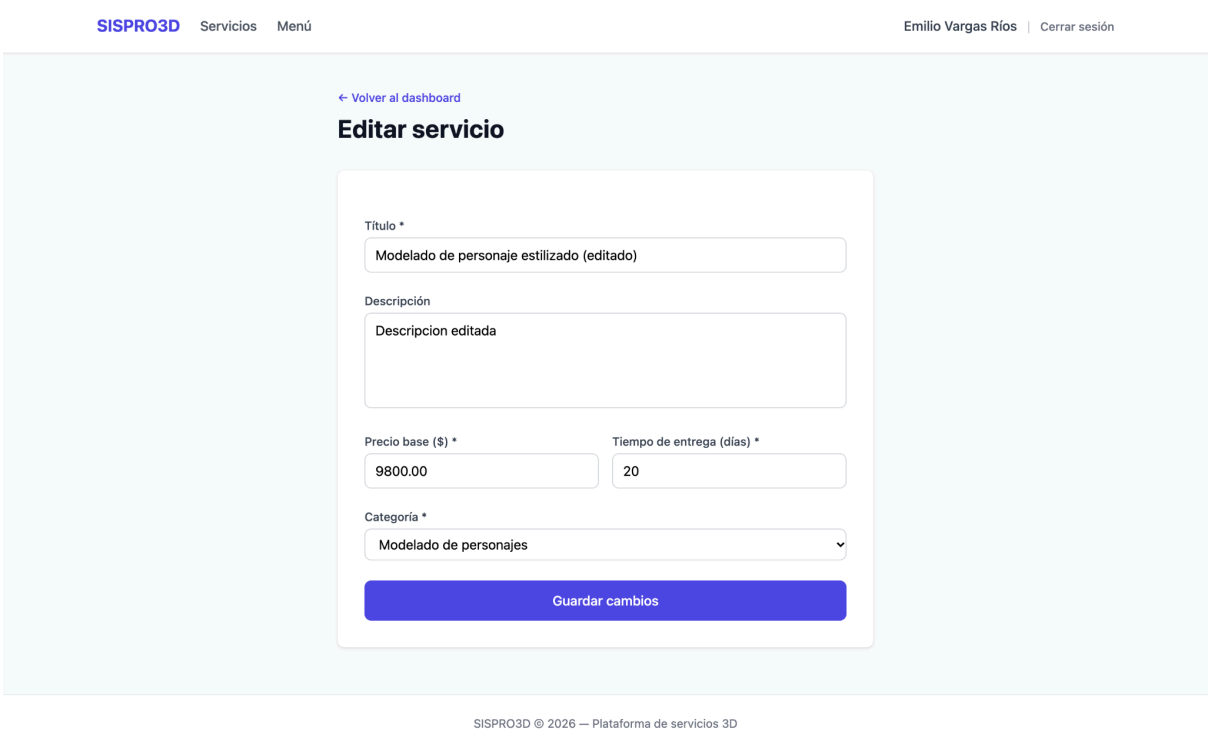


Figura 11.18: Interfaz de edición de servicio (los cambios reenvían la publicación a revisión en estado pendiente).

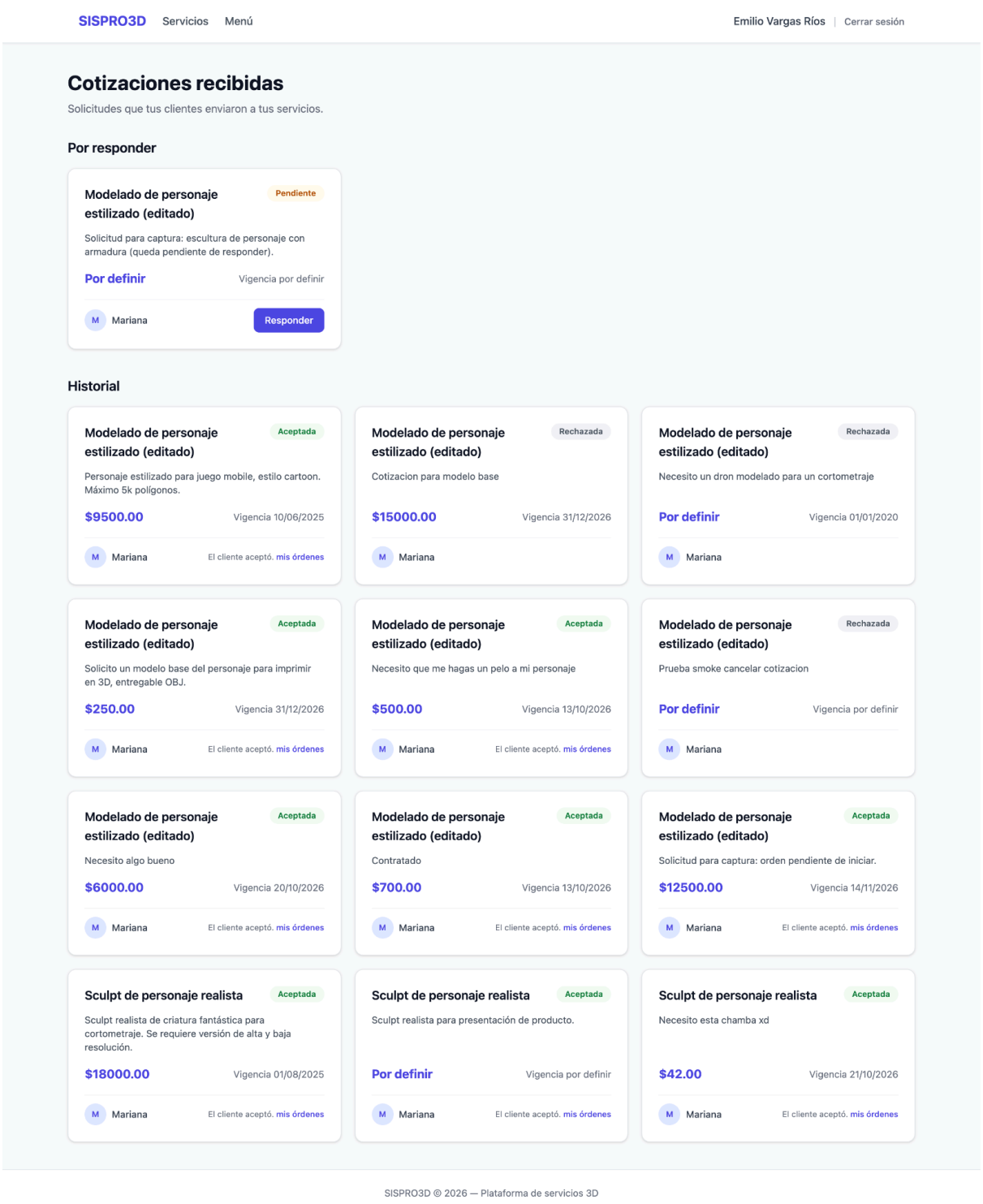


Figura 11.19: Listado de cotizaciones recibidas por el experto para la atención de solicitudes de trabajo.

Axel Fernando Montiel Aviles
Universidad Nacional Autónoma de México

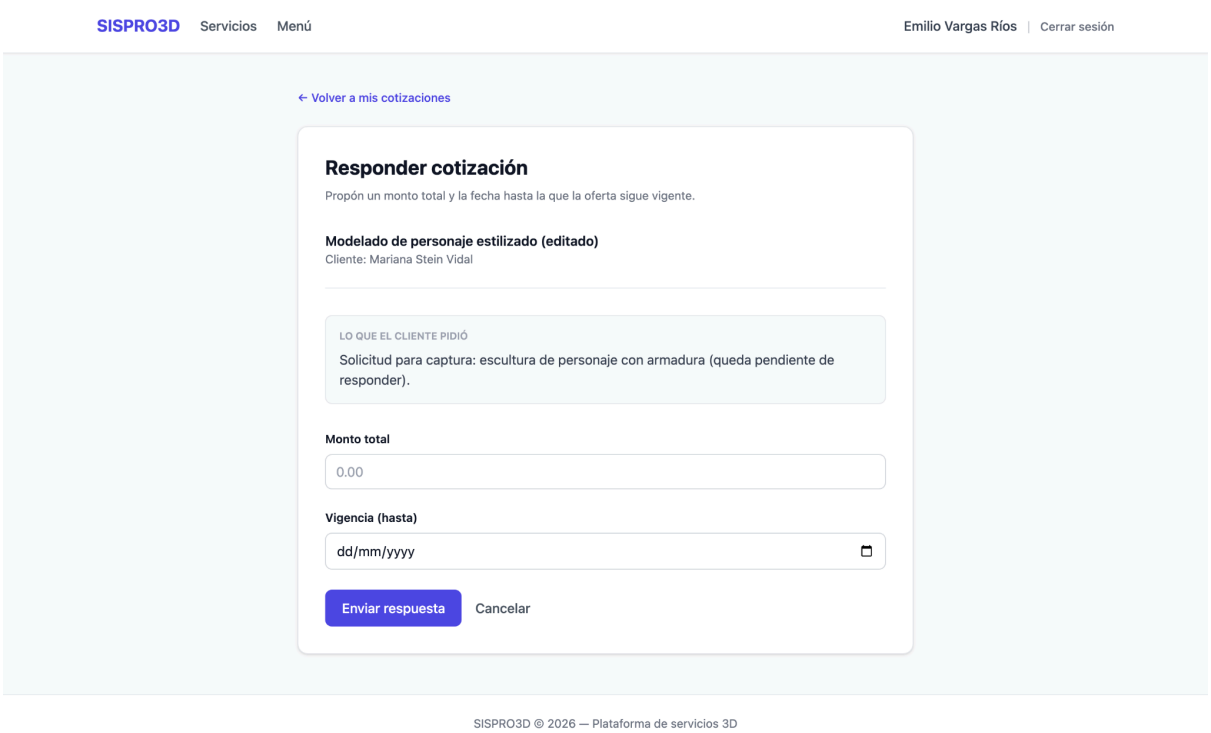


Figura 11.20: Interfaz para responder cotizaciones fijando el monto acordado y la fecha límite de vigencia de la oferta.

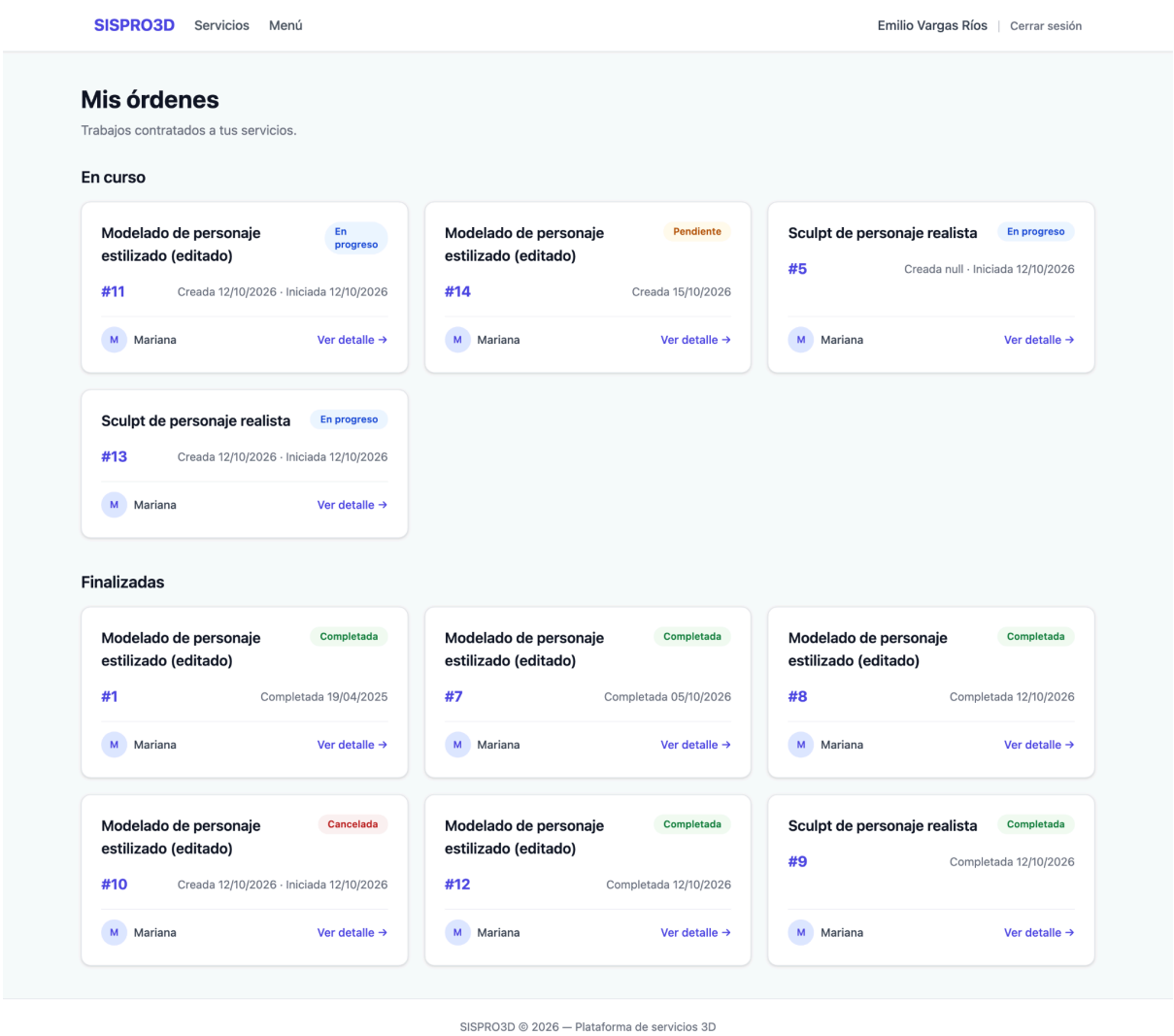


Figura 11.21: Panel de gestión de órdenes de trabajo recibidas por el experto indicando el estado del flujo.

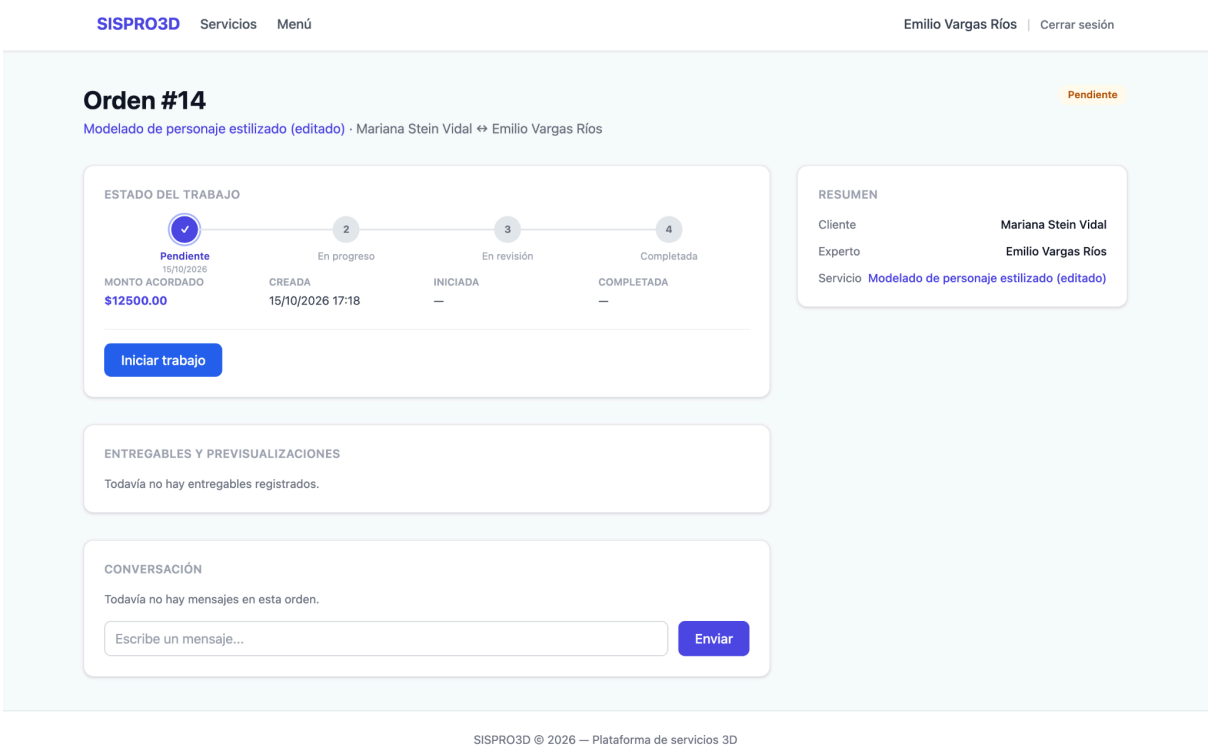


Figura 11.22: Vista de orden en estado pendiente con opción para que el experto inicie formalmente el trabajo.

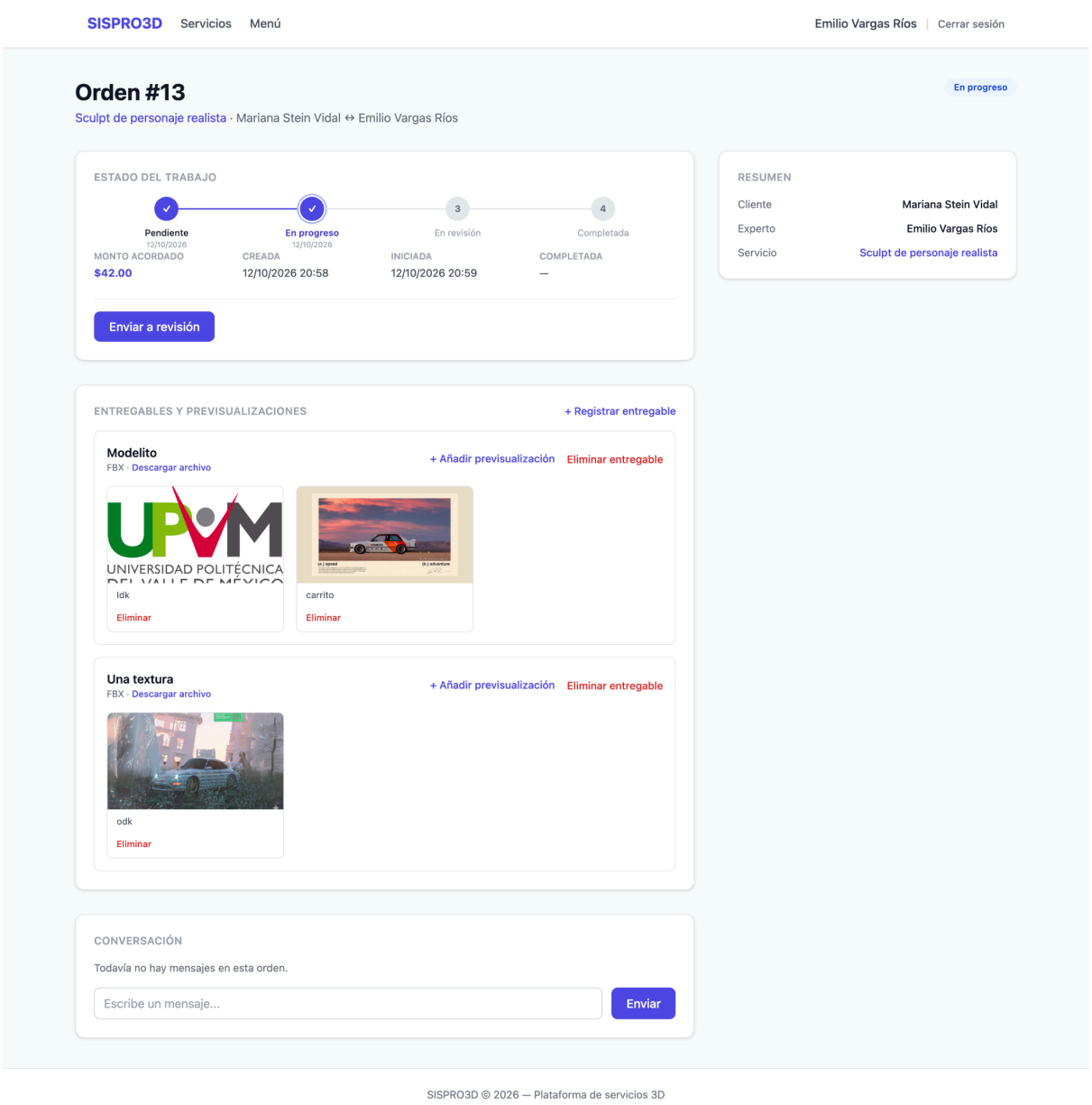


Figura 11.23: Panel de control del proyecto para la carga de entregables, previsualizaciones y gestión de revisión.

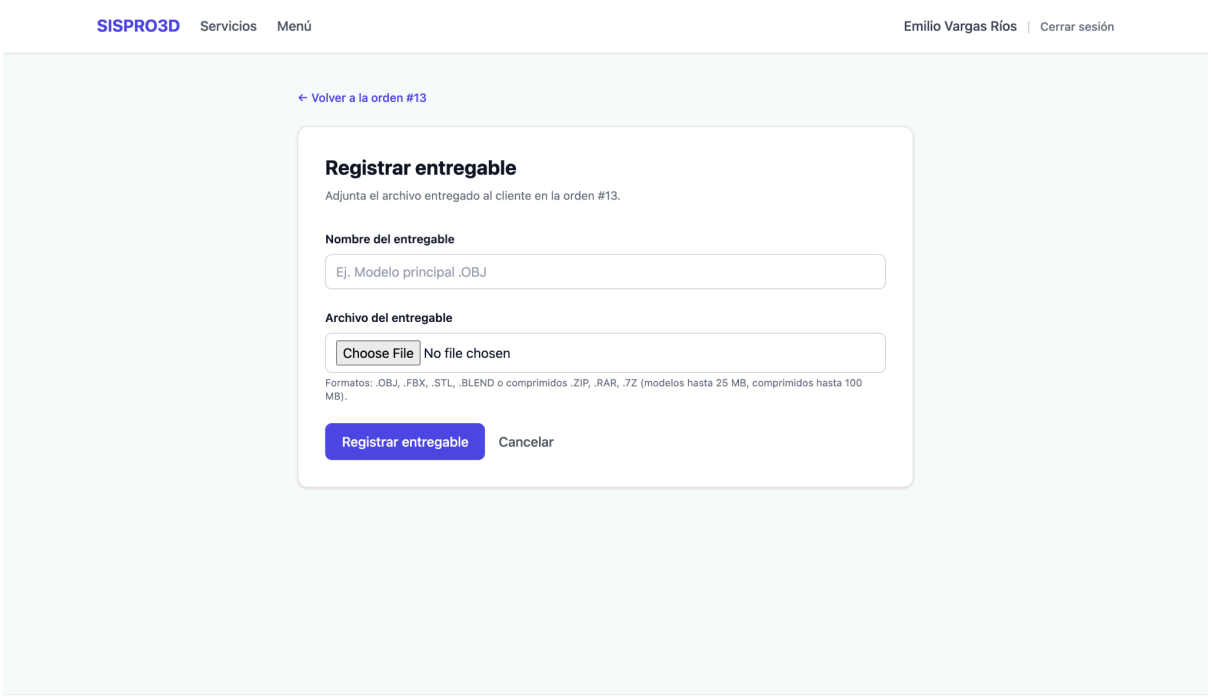


Figura 11.24: Formulario para la subida de archivos fuente y modelos 3D con restricciones de formato y tamaño.

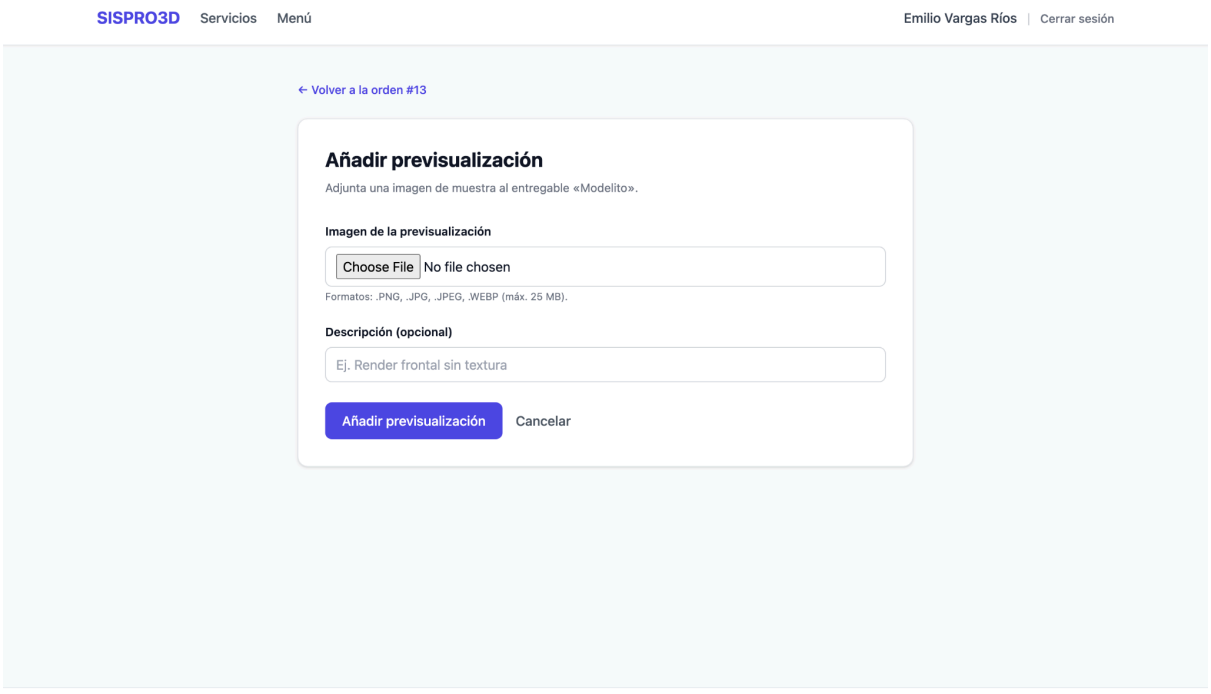


Figura 11.25: Interfaz para adjuntar imágenes de previsualización (renders) asociadas a un entregable específico.



11.4 Vistas Públicas y Acceso

Axel Fernando Montiel Aviles
Universidad Nacional Autónoma de México

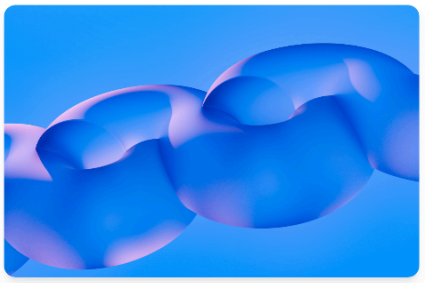
SISPRO3D Servicios Iniciar sesión Registro

MODELADO 3D PROFESIONAL

Da vida a tus ideas en 3D

Conecta con expertos en modelado, animación y diseño 3D. Publica tu proyecto, recibe cotizaciones y sigue tu pedido de principio a fin.

Explorar servicios | Crear cuenta



Servicios destacados

Una muestra de los servicios disponibles en la plataforma. Ver todo →

Modelado de personaje estilizado (editado) — Modelado de personajes Descripcion editada $9800.00 — 20 días de entrega	**Sculpt de personaje realista** — Modelado de personajes Escultura digital de alta resolución en ZBrush. Incluye retopología y mapas de desplazamiento. $18000.00 — 30 días de entrega	**Texturizado PBR completo** — Texturizado Texturizado realista con Substance Painter. Entrega de mapas Albedo, Normal, Roughness, Metallic y AO. $6500.00 — 10 días de entrega
Texturizado estilizado hand-painted — Texturizado Estilo hand-painted para modelos de juego. Inspirado en estilos tipo World of Warcraft o Fortnite. $5000.00 — 12 días de entrega	**Rigging y skinning de personaje** — Animación Configuración de esqueleto, pesos de skinning y controles de animación para personaje bípedo. $7500.00 — 15 días de entrega	**Animación de ciclo de movimiento** — Animación Paquete de animaciones: idle, caminar, correr, saltar y atacar. Entrega en FBX con 30fps. $12000.00 — 25 días de entrega

Explora por categoría

Encuentra el experto adecuado según el tipo de trabajo.

Modelado de personajes Creación de personajes 3D orgánicos o estilizados para juegos, películas o ilustración	**Modelado de entornos** Diseño y construcción de escenarios, props y arquitectura en 3D	**Texturizado** Creación de materiales, mapas UV y texturas PBR para modelos existentes	**Animación** Rigging, skinning y animación de personajes o elementos de escena
dad			

Cómo funciona

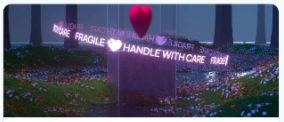
Contratar un servicio es sencillo.



1
Explora el catálogo
Encuentra el servicio que necesitas y revisa la ficha del experto.



2
Solicita tu cotización
Describe tu proyecto y recibe una propuesta del experto.



3
Recibe tu proyecto
Sigue la orden, revisa los entregables y valora el resultado.

¿Listo para empezar tu proyecto?

Explorar servicios

SISPRO3D © 2026 — Plataforma de servicios 3D

Axel Fernando Montiel Aviles
Universidad Nacional Autónoma de México

Figura 11.26: Landing page pública accesible para visitantes no autenticados en la plataforma.

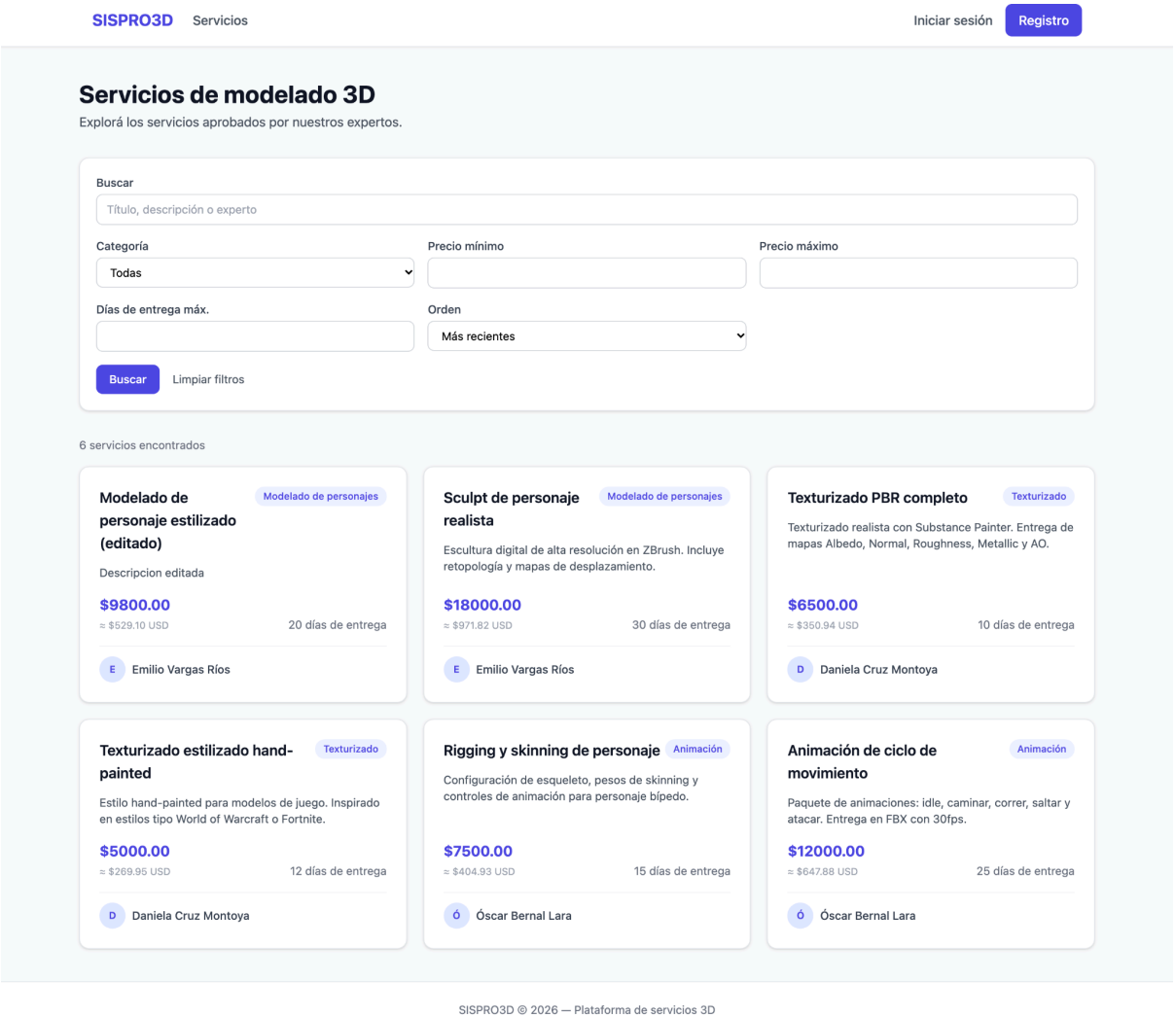


Figura 11.27: Vista de catálogo público con motor de búsqueda y filtros por categoría, precio y tiempo de entrega.

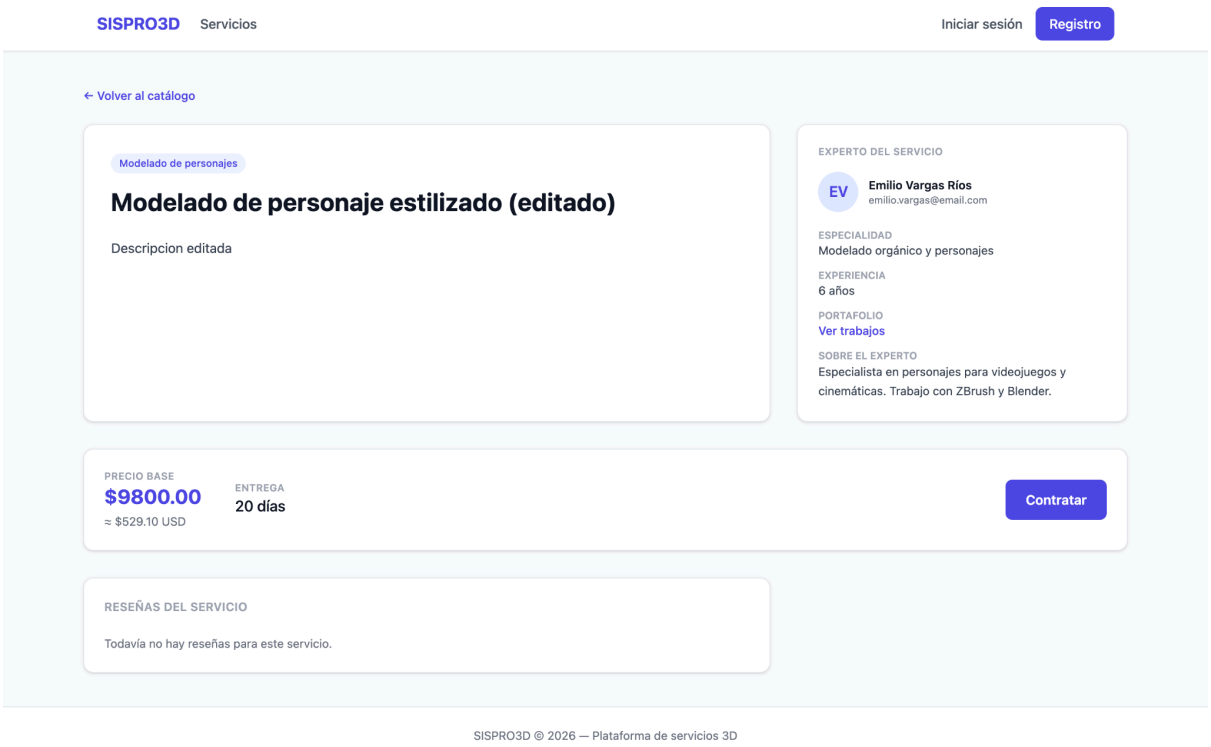


Figura 11.28: Ficha pública del servicio mostrando detalles generales e integración del tipo de cambio MXN/USD.

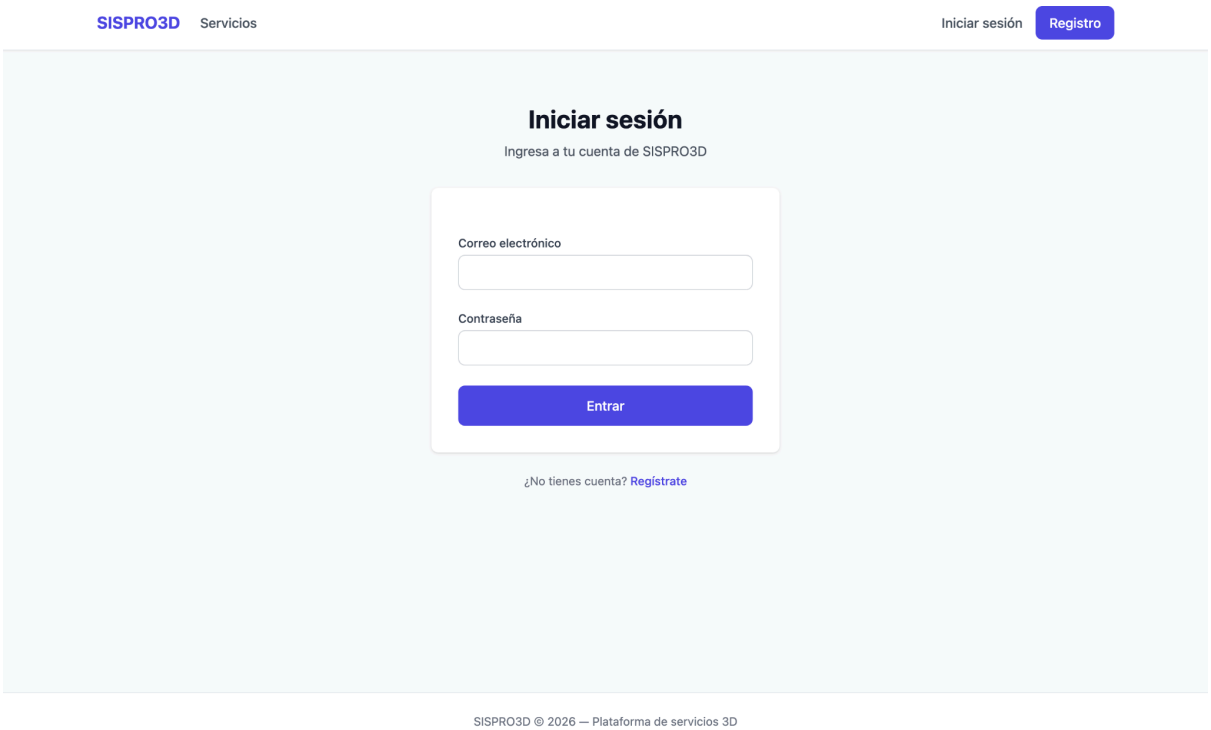


Figura 11.29: Formulario de inicio de sesión con autenticación basada en credenciales.

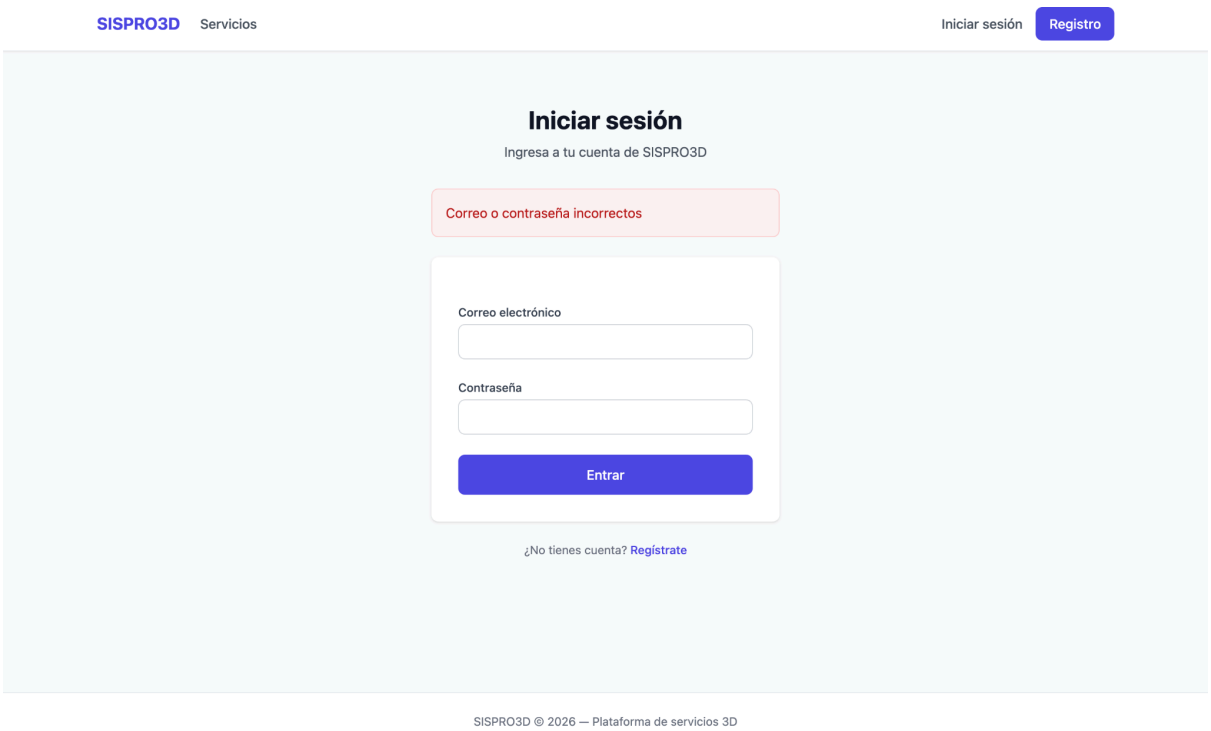


Figura 11.30: Manejo de errores de autenticación con mensaje genérico por seguridad.

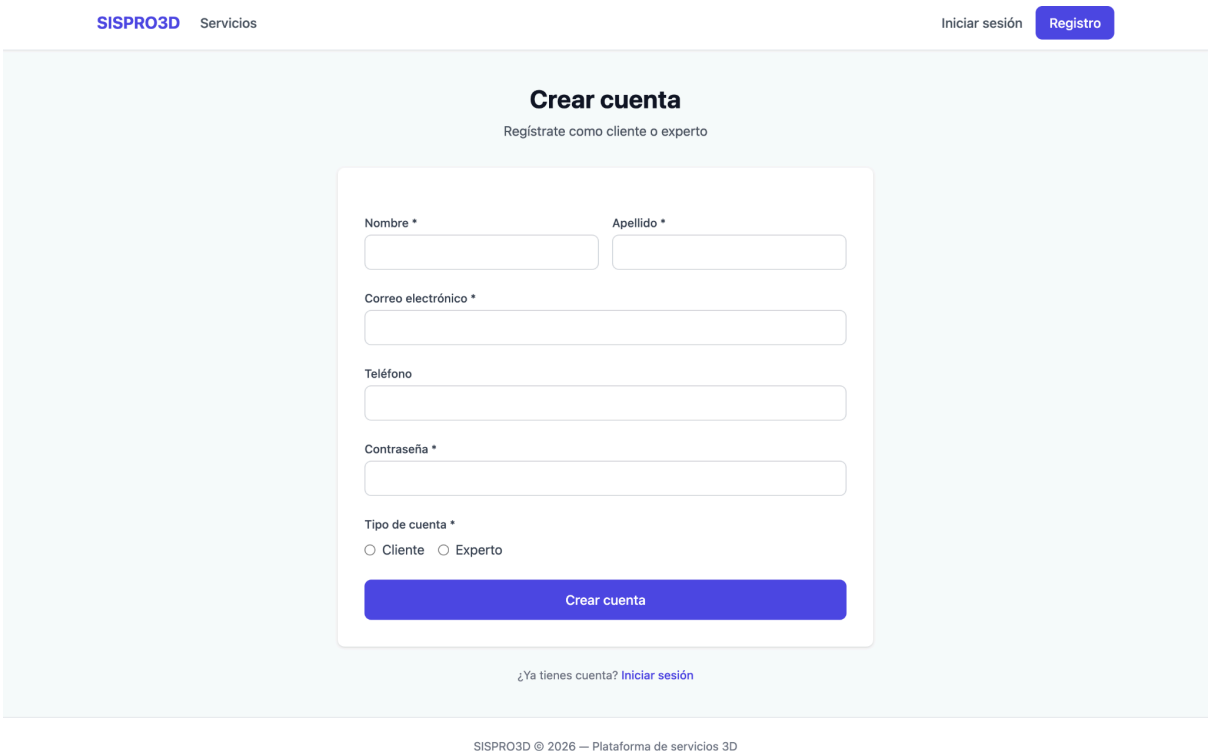


Figura 11.31: Formulario de registro de nuevas cuentas con selección de perfil Cliente o Experto.

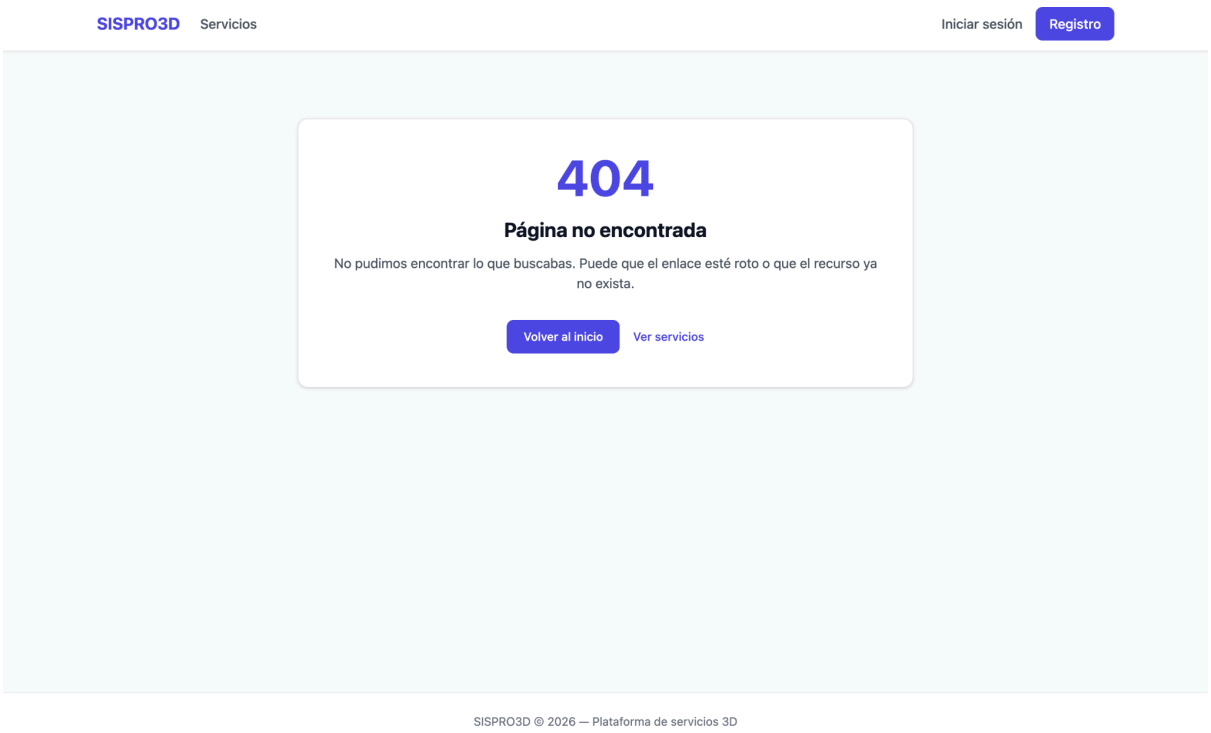


Figura 11.32: Vista personalizada para la captura de errores HTTP 404 de recursos no encontrados.

12. Código Fuente

- **Repositorio Git:** https://github.com/Blairi/SISPRO3D
- **Documentación en Markdown** Readme en el repo: https://github.com/Blairi/SISPRO3D

Instrucciones de despliegue

- **Requisitos previos del entorno:**
  - Disponer de Kit de Desarrollo de Java (JDK 21), Apache Maven 3.9+ y el motor de base de datos MariaDB 10.6+ instalado.
  - Verificar que la instancia del servicio relacional de MariaDB se encuentre activa y escuchando en el puerto local `localhost:3306`.
- **1. Configuración Inicial del Motor de Base de Datos:**
  - Ejecutar las siguientes sentencias SQL para inicializar el esquema y los permisos correspondientes:

```sql
CREATE DATABASE sispro3d_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'appuser'@'localhost' IDENTIFIED BY 'MiPasswordSegura123!';
```

```
GRANT ALL PRIVILEGES ON sispro3d_db.* TO 'appuser'@'localhost';
FLUSH PRIVILEGES;
```

    - El esquema DDL (`src/main/resources/schema.sql`) se aplica automáticamente durante el arranque del sistema (impulsado por las propiedades `spring.jpa.hibernate.ddl-auto=update` y `spring.sql.init.mode=always`). Para la carga inicial de registros de prueba, ejecute el script por única ocasión mediante la consola del motor:

Shell
```
mariadb -u appuser -p sispro3d_db < scripts/data.sql
```

- **2. Parametrización de Propiedades de la Aplicación:**
    - Los parámetros principales del sistema se gestionan centralizadamente desde el archivo `src/main/resources/application.properties`:

Parámetro / Propiedad	Valor predeterminado asignado
`spring.datasource.url`	`jdbc:mariadb://localhost:3306/sispro3d_db?useSSL=false&serverTimezone=UTC`
`spring.datasource.username / .password`	`appuser / MiPasswordSegura123!`
`server.port`	`8080 (puerto predeterminado)`
`app.jwt.secret`	`${SISPRO3D_JWT_SECRET:dev-only-signing-key...}`
`app.storage.location`	`uploads`
`external.exchange-rate.*`	`Proveedor externo Frankfurter (https://api.frankfurter.dev)`

    - Para ambientes de despliegue en producción, estas propiedades deben ser sobrescritas por medio de variables de entorno del sistema operativo (`SISPRO3D_JWT_SECRET`, `SPRING_DATASOURCE_URL`, `SPRING_DATASOURCE_USERNAME` y `SPRING_DATASOURCE_PASSWORD`), evitando la modificación directa del código fuente.
- **3. Compilación y Ejecución del Proyecto:**
    - Para construir el paquete ejecutable del sistema omitiendo las pruebas unitarias:

```shell
mvn clean package -DskipTests
```

    - Iniciar la aplicación desde el paquete compilado JAR:

```shell
java -jar target/SISPRO3D-1.0-SNAPSHOT.jar
```

    - De forma alternativa, se puede arrancar el entorno de desarrollo mediante el plugin de Spring Boot:

```shell
mvn spring-boot:run
```

    - Una vez completado el despliegue, la plataforma web estará accesible en la dirección local `http://localhost:8080`.
- **4. Verificación de Componentes y Puntos de Acceso:**
    - **Interfaz Web Protegida (Thymeleaf):** http://localhost:8080/login (las credenciales de prueba se encuentran documentadas en el archivo README).
    - **Catálogo Público de Servicios:** http://localhost:8080/services.
    - **Interfaz API REST Protegida:** `http://localhost:8080/api/v1/**` (autenticación sin estado mediante tokens JWT; inicio de sesión disponible en la ruta pública `POST /api/v1/auth/login`).
- **5. Consideraciones para Entornos de Producción:**
    - Sustituir de forma obligatoria la clave de firma JWT y las credenciales de la base de datos por valores complejos encriptados.
    - Garantizar que el directorio especificado en `app.storage.location` (`uploads/`) exista físicamente en el servidor de aplicaciones y disponga de permisos de lectura y escritura.
    - Si bien la estrategia `spring.jpa.hibernate.ddl-auto=update` facilita el desarrollo, se recomienda implementar scripts de migración versionados en entornos de producción reales.

  El sistema no requiere dependencias obligatorias de CDNs o infraestructura externa de terceros; todos los recursos estáticos se sirven de forma local y el módulo de integración externa con la API de tipo de cambio opera con un mecanismo de tolerancia a fallos.

13. Conclusiones

Con el desarrollo de SISPRO3D logramos integrar en un solo proyecto varios de los conocimientos que hemos adquirido en el diplomado de desarrollo de aplicaciones con Java y el ecosistema de Spring. A lo largo del proyecto trabajamos con Spring Boot, Spring Data JPA, Spring Security, Thymeleaf, APIs REST y MariaDB, aplicándolos sobre un problema de negocio concreto y no solamente como ejercicios aislados. Esto nos permitió entender mejor cómo se relacionan las diferentes capas de una aplicación y cómo llevar una idea desde los requerimientos hasta una implementación funcional.

Uno de los principales aprendizajes fue la importancia de definir correctamente la arquitectura y las reglas de negocio antes de implementar las funcionalidades. La separación por capas y módulos nos permitió mantener responsabilidades claras entre controladores, servicios, repositorios y entidades, haciendo que el sistema sea más fácil de entender y mantener. También nos ayudó a trabajar de una forma más cercana a lo que encontramos en proyectos reales de desarrollo.

En cuanto a seguridad, implementamos autenticación y autorización utilizando Spring Security y JWT, diferenciando los permisos de acuerdo con los roles de Administrador, Cliente y Experto. También incorporamos validaciones, manejo centralizado de excepciones y respuestas HTTP estandarizadas, lo que nos permitió tener un mayor control sobre los diferentes escenarios que pueden presentarse durante la ejecución del sistema.

Otro punto importante fue el desarrollo de la API REST. Logramos definir los recursos y operaciones principales del sistema, documentando 62 endpoints y estableciendo una estructura uniforme para las respuestas y errores. Además, utilizamos pruebas de integración con una base de datos H2 en memoria, lo que nos permitió probar diferentes funcionalidades sin depender directamente de una instancia física de MariaDB.

También aprendimos que una aplicación no debe depender completamente de servicios externos para funcionar correctamente. En la integración con el servicio de conversión de moneda implementamos tiempos de espera, caché y una estrategia de degradación, de manera que si el servicio externo no está disponible, la aplicación puede continuar funcionando mostrando la información principal.

Finalmente, considero que SISPRO3D me permitió acercarme a una forma de desarrollo más parecida a un proyecto profesional. Más allá de implementar las funcionalidades solicitadas, tuvimos que considerar aspectos como seguridad, persistencia, manejo de errores, pruebas, documentación, integración con servicios externos y mantenibilidad. Estos conocimientos nos sirven como una base para seguir



creciendo profesionalmente y para enfrentarnos a proyectos de mayor complejidad utilizando tecnologías con las que ya comenzamos a familiarizarnos en nuestra etapa profesional.

Axel Fernando Montiel Aviles
Universidad Nacional Autónoma de México